Introduction

Welcome to the course! To fully understand and appreciate blockchain, there are multiple interdependent topics you need to understand together. Some foundations need to be built so the terminology can make sense. This course will slowly build those foundations. By the end of this course, you will be able to understand and run a hyperledger fabric network, deploy your chaincode and interact with it.

Let’s begin with a basic introduction to blockchain. There are some gaps in this introduction that will be filled sometime in the next 2-3 chapters so please give it some time and things will start to come together.

Its really by the end of the 4th chapter when you will be able to understand blockchain and its potential to revolutionize the world.

In the next lesson, we will discuss some distinctive features of blockchain.

What is Blockchain?

The key features of Blockchain technology

WE'LL COVER THE FOLLOWING

- Immutability
- Distribution

Blockchain is a term used to describe **DLT**, or **Distributed Ledger Technology**. Blockchain is used to create a storage system for data in a distributed and immutable manner.

These are the key features from a technological standpoint.

Immutability

This means that once data is written to a blockchain data store or **ledger**, it cannot be changed – its there forever. In contrast, in a standard Relational Database, no matter how much security you implement, the data can be accessed and modified on the file system on which the data is persisted. This could be done by corrupt admin or a hacker.

A blockchain system ensures that even if a bit of data is changed at any level on the ledger, the entire system will report an `invalid` state. And since the data is `distributed` on multiple systems, the actual data with a valid state can be recovered from one of the systems.

Distribution

As long as you see data on a blockchain and its in a `valid` state on a majority `distributed` nodes, you can trust that data to be accurate. This *trust* is key. This trust is achieved in a blockchain system by replicating the datastore on a number of peers(hosts) on the internet. If one of the misbehaving peers goes in an `invalid` state, the other peers can filter it out. As long as there is a

majority of peers agreeing to a common `valid` state, you can completely trust the data that is stored on that system. This replication also guarantees high availability.

This trust is vital! No other system in the past has been able to develop this by design. In this course we will see how blockchain provides this trust *by design*.

Test Yourself

1 Which best describes an immutable record?

COMPLETED 0%

1 of 3

First, lets examine how current systems struggle with trustworthiness and the costs incurred as a result. In the next lesson we will look at an example from the automobile industry.

Lack of Trust in Current System of Records - Automobile Industry

We will look at the issue of trustworthiness in the automobile industry and how blockchain can solve this problem.

WE'LL COVER THE FOLLOWING

- Trust Issues in the Automobile Industry
- Resolving trust issues using Blockchain

Trust Issues in the Automobile Industry

I recently imported a used car from another country. If you have bought a used car at some point in your life, you know the trouble and the confusion you must have gone through in order to be able to 'trust' the condition of the car (mileage, service history, accident reports, etc.).

There are billions of dollars lost every year due to odometer tampering. In some cases, people have bought a used car and later found there to be a major accident on the car.

On the other hand, as a seller of a used car, if my car is extremely well kept I am not able to fully convince the buyer to get the right value for my well-kept car.

In countries like the US, there are also entities like Carfax, a middleman, which people 'trust' to some extent as a keeper of their data. This trust is put into Carfax because there are legal processes in place to hold them accountable if they were to alter the data for any car.

For car service records, we have paper-based car service books, which can, of course, very easily be faked. If one was to take a paper based service record and reconcile it with 10 different service providers to ensure validity of the data, it would be very time-consuming, since all service providers have their

own system of records.

Resolving trust issues using Blockchain

Now imagine the following scenario, all the service providers are pushing their service records to a blockchain and the mileage readings of car is being pushed on the blockchain by service providers & IoT devices in car. Moreover, insurance companies are pushing accident details of the car on blockchain. All the timestamped records function to reliably record what happened to the car and by whom. It would be a huge peace of mind for a customer, buying a car with such 'trustable' record. Insurance companies would benefit immensely as well, when looking at that data and deciding insurance premiums for a car. And if we were to maintain a mapping of cars to their owners on blockchain, insurance companies benefit further by studying driver behavior.

Test Yourself

Q

What feature of blockchain technology is best illustrated by the used car example?

COMPLETED 0%

1 of 1

In the next lesson, we will discuss the cost of trust in the financial industry.

Cost of Trust in the Financial Industry

Time is money

Let's take a look at another example.

Let's say you need to send $100 to a friend in the UAE.

You contact your bank, and they debit that amount from your account. Your bank is regulated by the state bank (the central authority) which is why you trust your bank to send it as promised.

Now, there are multiple ways in which a cross country transfer is processed by banks. Lets look at the most basic one.

Your bank updates its records and sends it to the central bank. The central bank updates its record and sends it to the UAE central bank, which sends it through the central bank of that country. They again update their record and send it to your friends bank. Your bank in the UAE then credits his account. It takes days but eventually your friends can access that money.

And this is just the process for 'record' updating, where your account is debited and your friends' is credited. The central authorities of both countries update their records to keep track of how much money must be moved from one end to the other. The actual movement of money might happen later in a different form, carrying its own long process of record updating and trust development.

In this process, multiple employees from all these banks are involved for safe updating of records. Because the stakes are so high, things are done safely to ensure trust is developed. At the same time, the $100 you paid will incur some processing fee deductions at all stops along the way. The trust you put in the current financial system is paid through your time and money. This is where bitcoin comes is as an alternative to this system! In this new system, both you and your friend have a bitcoin address. All you need to do is transfer the

money to your friend's address. No fee, no delay, no middle parties, no cost buying 'trust', as the system is trusworthy be design. . We will see in a later chapter with technical detail how bitcoin achieves this. In the next lesson, we will again discuss how current systems struggle with trustworthiness but this time, we will be using a supply chain example.

Test Yourself

Q

Owing to which design feature is bitcoin able to avoid huge transfer fees?

COMPLETED 0%

1 of 1

Lack of Trust in Current System of Records - Supply Chain

Issue of trustworthiness in the supply chain industry and how blockchain can solve this problem.

Lets take a very simple example without cryptocurrency.

Let's imagine you are going to buy a pack of frozen vegetables from your grocery store. Lets also imagine you are in a third-world-country where food regulations by a central authority are weak. How well do you trust that the pack of vegetables has been kept in the required temperature range from the time it was cut and preserved to today? You ensure you are buying from a big, well-reputed grocery store. You ensure that the vegetable pack comes from a good trusted brand. But do you fully trust any of this, given there are numerous instances where big companies like Nestle have screwed this up?

Would you trust them if they simply showed you a record of temperature readings in their 'own database' to show that the temperature guideline has been met? Would you trust the database admin, the IT team, the maintainers of the data?

Lets imagine another scenario. The vegetables are cut and packaged and a record is added on the blockchain and written on hundreds of peers. Once that package is stored an IoT device repeatedly logs the timestamped temperature entries on the blockchain(on distributed peers). When the item is shipped, the shipping container's are fitted with IoT logging the timestamped temperature readings on blockchain. At all points of the supply-chain data is read by sensors and put on blockchain - signed and timestamped.

Since you can access the data from blockchain for your vegetable pack and see that it has always remained in the safe zone you can feel safer. You can trust the data coming out of blockchain, without needing a food regulation authority.

Test Yourself

Q

What feature of blockchain design was best illustrated by the frozen vegetables example?

COMPLETED 0%

1 of 1

In the next lesson, we will explain the different terminologies used so far.

Terminology

In this lesson, we will explain the different terminologies used so far in this chapter

Since there are multiple terms being used here lets clarify some so there is no confusion:

The data store that keeps all blockchain data is called a **ledger**. Each entry in the ledger is called a **transaction**. The transactions are timestamped and stored in groups of **blocks**.

On the ledger you can only add a new transaction or read the old ones. You can not update or delete anything.

The **ledger** is designed to be immutable using cryptographic algorithms (we will study in next chapter) and its distributed design. The **ledger** is replicated and maintained by multiple **hosts** or **peers** or **nodes**. This is why its called **DLT** or **Distributed Ledger Technology**.

In the next lesson, we will see a quick recap of the whole chapter.

Recap

What we know

WE'LL COVER THE FOLLOWING

- What?
- Why?
- Next: How?

To recap: we saw in this chapter the *what* and *why* of blockchain.

What?

Blockchain or Distributed Ledger Technology (DLT) is a 'trustable data record' built as a distributed system.

Why?

Current system of records either lack trust or pay a cost for trust. There is a huge opportunity to solve these issues in current systems of records.

Next: How?

In the next 2 chapters we will see from a tech perspective, how blockchain achieves this quality of 'immutability and trust'.

In the next chapter, we will explain the different pre-requisites for understanding blockchain.

Introduction

This introductory lesson provides a short introduction to this chapter.

Before we dive deep into blockchain design it's important to understand cryptography concepts on which blockchain is based. In this chapter we will cover those concepts. If you already have a clear understanding of these please feel free to skip this chapter and just jump to its recap section.

So lets start by understanding **hash functions**.

In the next lesson, we will discuss hash functions.

Hash Functions

A hash function maps data of arbitrary length to a unique fixed length string, called digest.

WE'LL COVER THE FOLLOWING

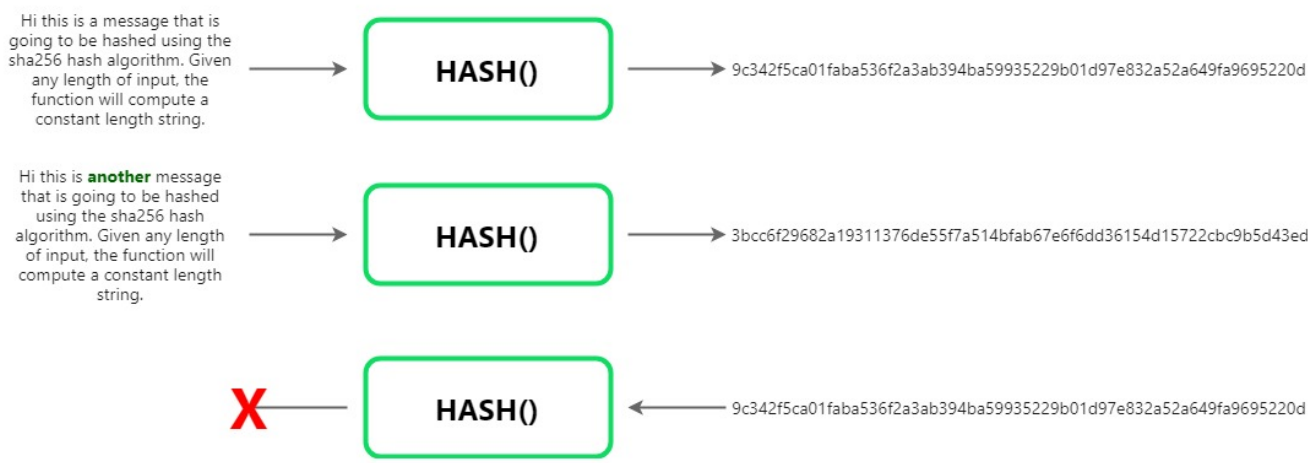
- Hash Functions for Checking Integrity
- SHA256 Hash Function

A hash value for a data is X is a function:

`HASH(X) = Y`

Such that:

- No other X' can have HASH(X') equal to Y. Its one to one mapping.
- The size of Y is fixed and the size of X can be arbitrary.
- Given Y you can not calculate X. Its a one-way function!



Hash Functions for Checking Integrity

This means that I can take a huge text file and compute its unique digest using a hash function. If I send that file and its computed hash along with it to a receiver, Bob, Bob can then recompute the hash to ensure that the content of that file were not corrupted in the transmission. When we download a file

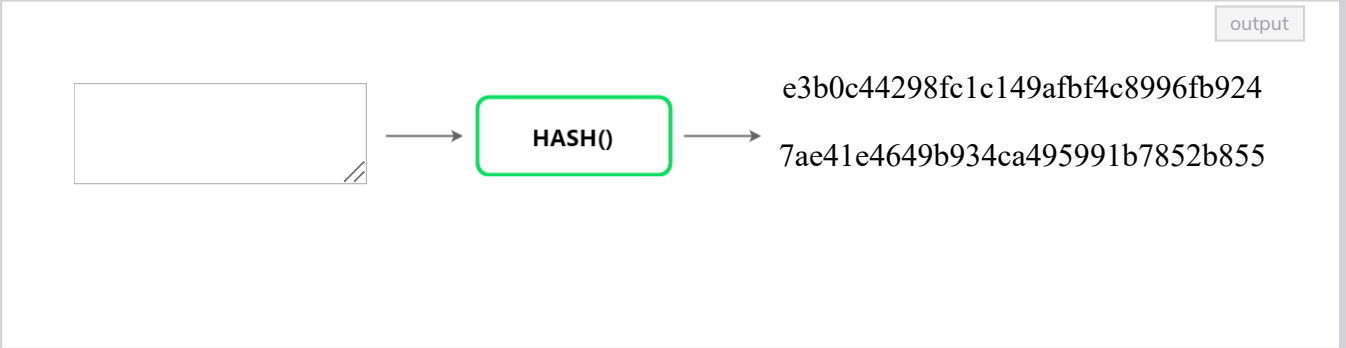
that file were not corrupted in the transmission. When we download a file from the internet, it uses the same hash functions to verify its integrity.

SHA256 Hash Function

There are multiple standardized hash function implementations that are used, such as SHA256, which we will be using in our course.

You can find libraries that implement SHA256 hash in all technologies, so you never have to write your code for SHA256 implementation.

Here is a small Javascript widget that will calculate sha256 of any data you enter:



Test Yourself

1 Which of the following statements is true?

COMPLETED 0%

1 of 3

In the next lesson, we will discuss what Public key cryptography is.

Public Key Cryptography

In this lesson, we will explain what public key cryptography is and it's key usage.

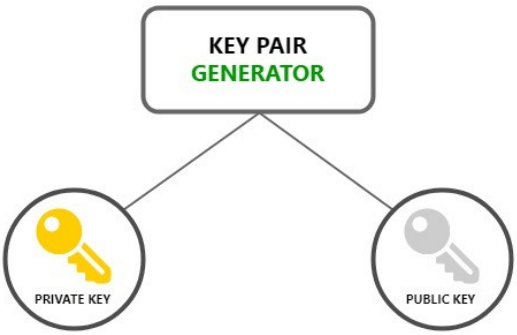
WE'LL COVER THE FOLLOWING

- What is public key cryptography?
- Key usage
- Key generation playground

What is public key cryptography?

Public key cryptography is a cryptographic system used for the encryption/decryption of data. It is not one way like a hash function, meaning that the data, once encrypted, can be decrypted if you have the required key.

You start by generating a special, related pair of keys. These keys can be generated in a pair only. They are output from single execution of a key generation algorithm.



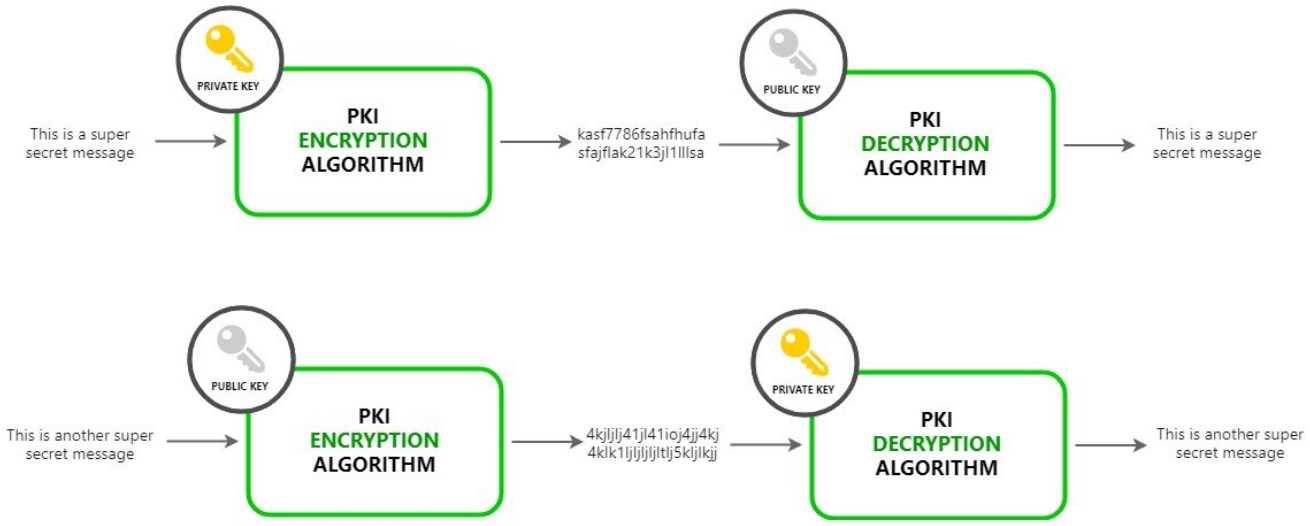
Color coding in this course:

- Golden key: private key
- Silver key : public key

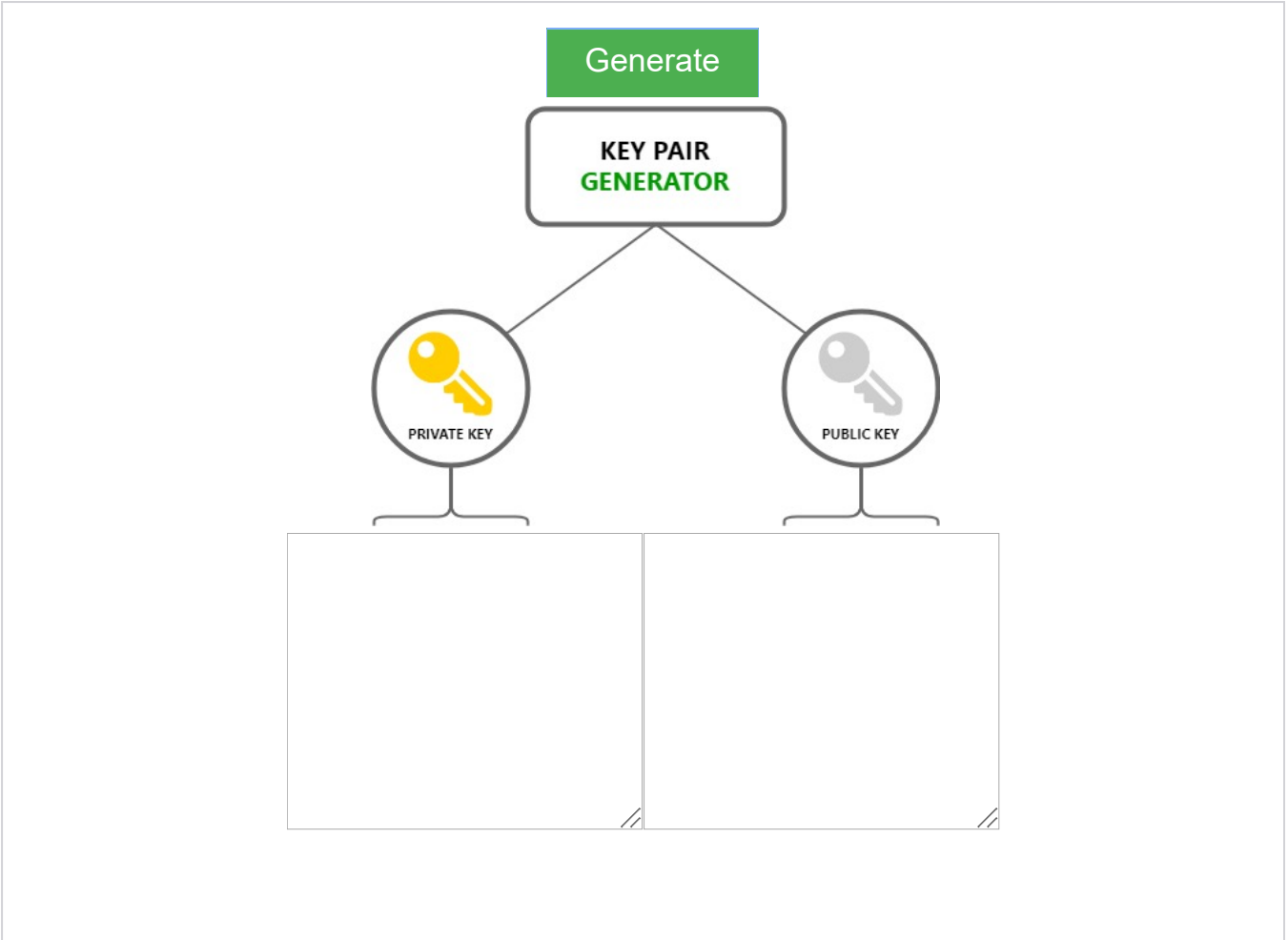
Key usage

Any key can lock or encrypt the data. To unlock or decrypt we need the other key. The only way to decrypt and make the data readable is by having the corresponding key.

In the next lesson, we will show how this cryptographic technique is used to ensure communication security.



Key generation playground



Test yourself

1

If a person possesses a private key, the person may generate the corresponding private key using a key-pair generator.

COMPLETED 0%

1 of 2

In the next lesson, we will show how this cryptographic technique is used to ensure communication security.

Secure Communication using Public Key Cryptography

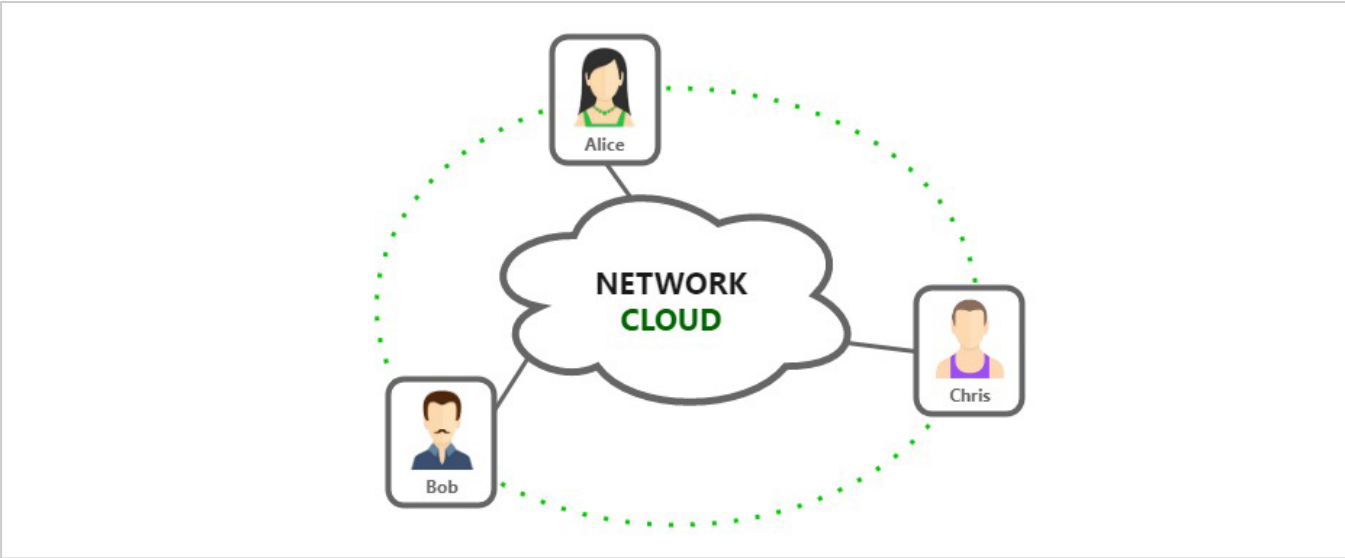
In this lesson, we will see how two parties can communicate securely via public key cryptography.

WE'LL COVER THE FOLLOWING

- How can we transmit a secret message using PKC?

Now that you understand the basic features of asymmetric key pairs, let's see how this is used to ensure that two or more parties can communicate private messages on an open and insecure network.

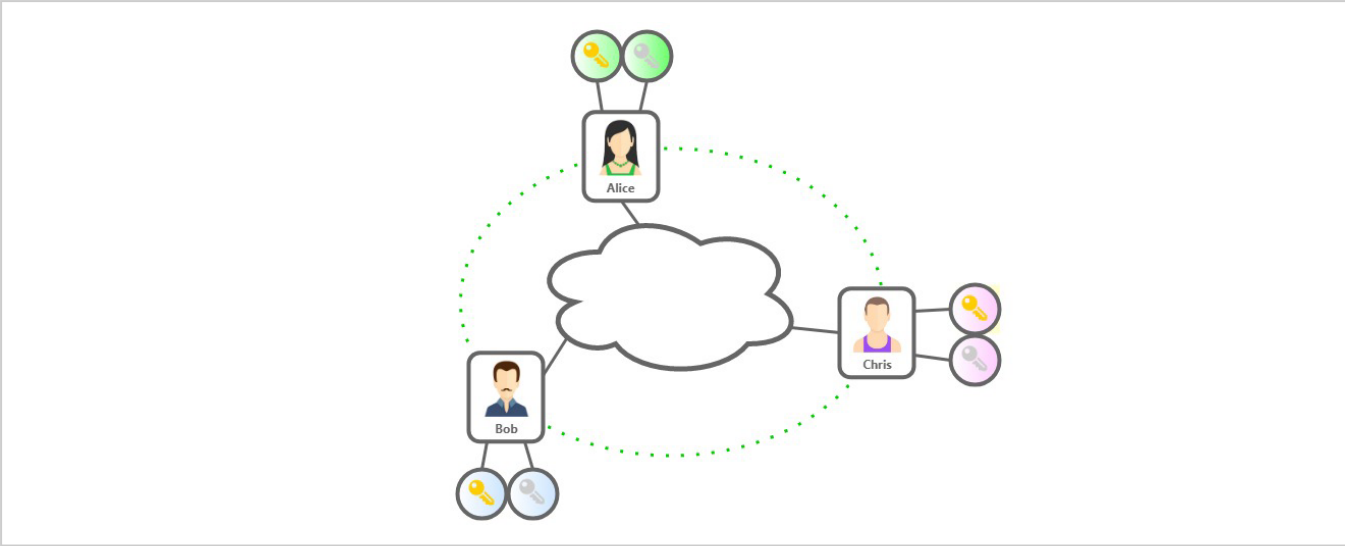
Lets say Alice, Bob and Chris are connected through a network on which everyone can read data being transferred. If Alice was to send a secret message to Bob only, it would not be possible as Chris gets to read everything transmitted through the network.



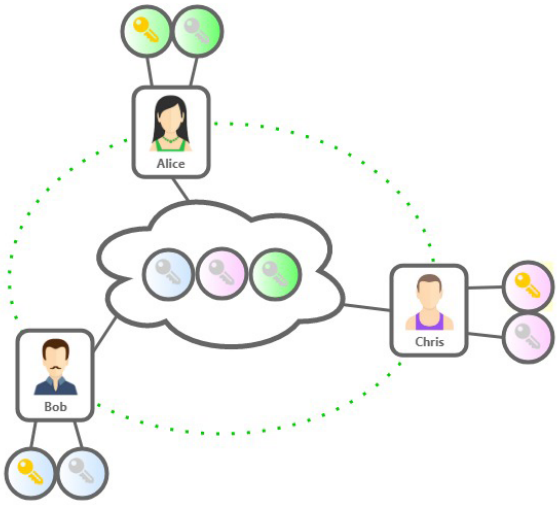
Insecure network

How can we transmit a secret message using PKC?

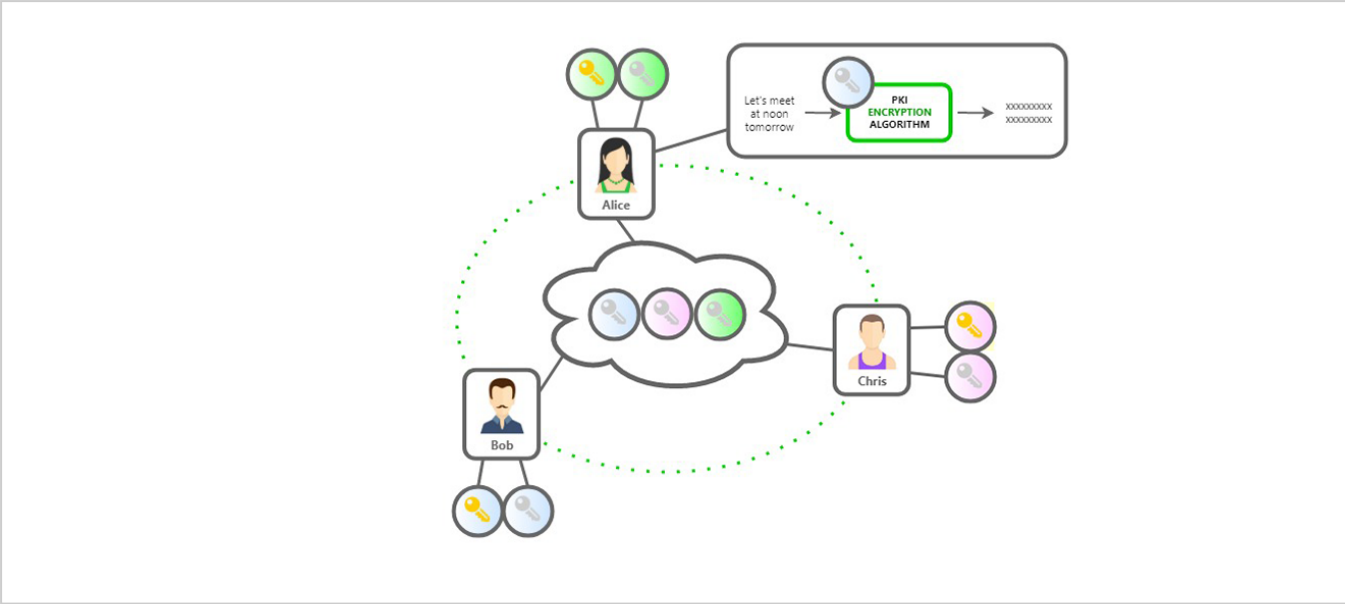
1. Each participant generates their own key pairs.



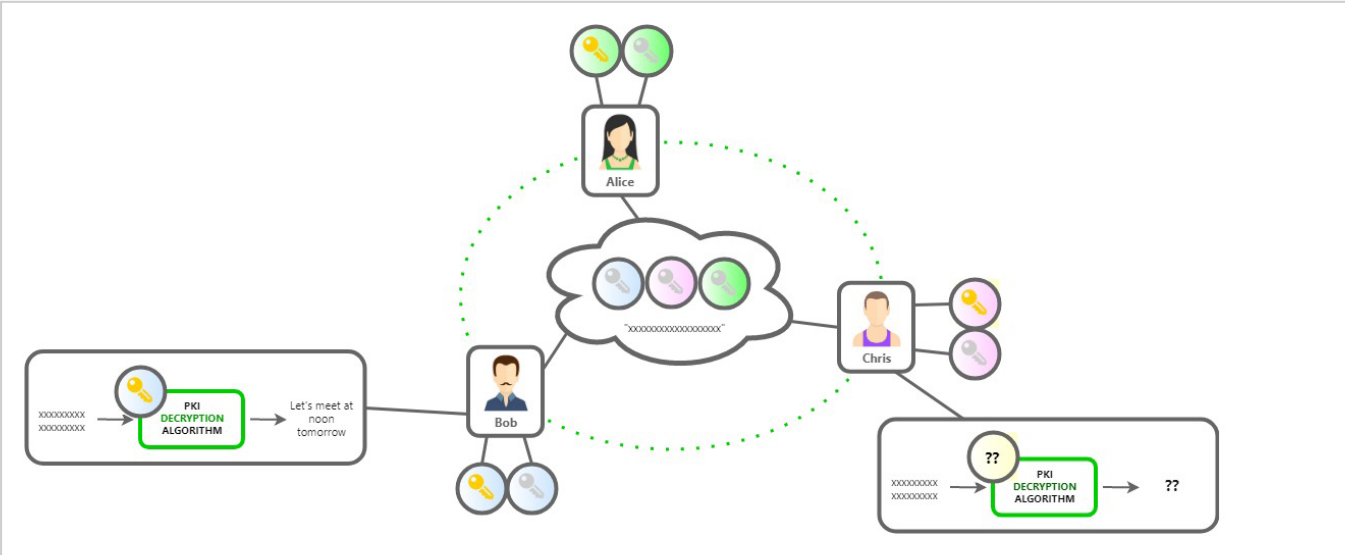
2. Each party **guards their private key** and **broadcasts the public key** on the network.



3. Now let's say Alice wants to send a message to Bob saying "Let's meet at noon tomorrow" but she does not want Chris to know about it. Alice will encrypt the message with Bob's public key and share through the network.



4. Since the corresponding private key is only known to Bob, only Bob can decrypt it. Chris cannot make any sense of the encrypted message.



No Image selected

1 The above example works on what principle of public key cryptography?

In the next lesson, we will see how to add digital signatures using public key cryptography.

Digital Signatures using Public Key Cryptography

In this lesson, we will see how to ensure the recipient of one's identity using public key cryptography.

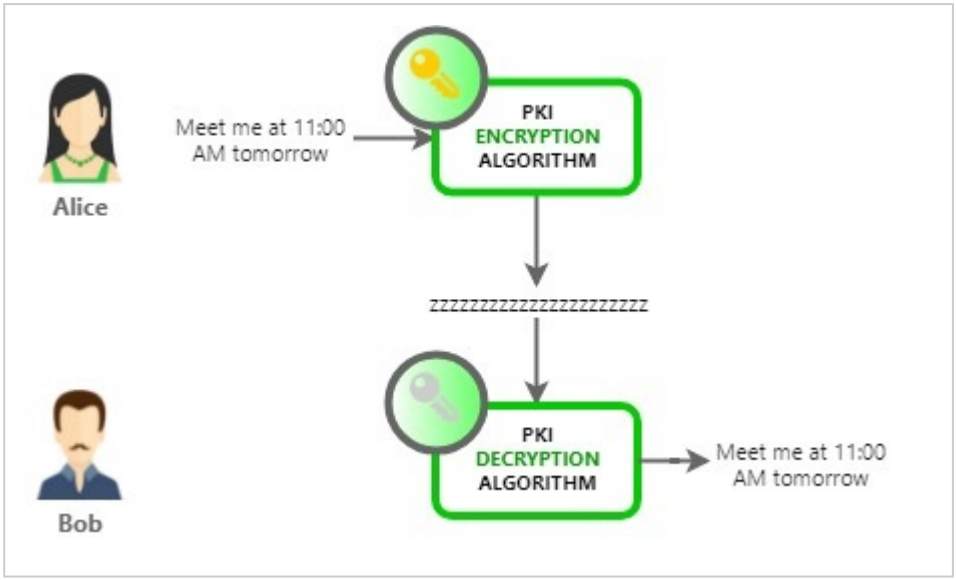
WE'LL COVER THE FOLLOWING

- Adding a Digital Signature
- Verifying a Digital Signature
- So far...

Let's continue with the previous example. Where Bob receives a message, which when he decrypts says "Meet me at 11:00 tomorrow". How can Bob know for sure that the message was sent by Alice and not Chris?

This can be achieved if the message is "signed" by its sender (Alice). The receiver of the message can then verify the signatures.

We need a way for Alice to "digitally sign" the message so the receiver of the message (Bob) can trust that the message was sent by Alice. Alice can add a digital signature to the message by using her private key to encrypt it. When the message is decrypted by her public key only, Bob can know for sure it came from Alice.

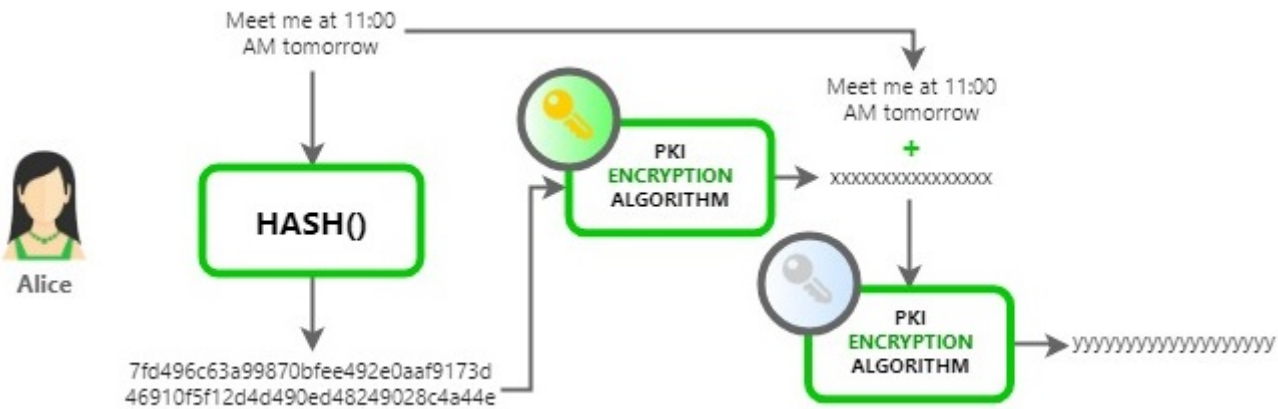


Alice sends digitally signed message to Bob. Bob can verify using Alice's private key

This works, but to make it more efficient, instead of signing the entire lengthy message the sender computes a hash (digest) of message and signs that with his public key instead. The receiver can then re-compute the message hash and compare it with the signed hash to ensure that message was not tampered with.

Adding a Digital Signature

1. Alice computes a message digest by hashing the message she is about to send. SHA256("Meet me at 11:00 tomorrow").
2. Alice calculates the message digest = HASH(message);
3. Alice signs the message digest by encrypting it with her private key.
4. Alice appends the signed digest with message and encrypts with Bob's public key Alice sends it over to Bob

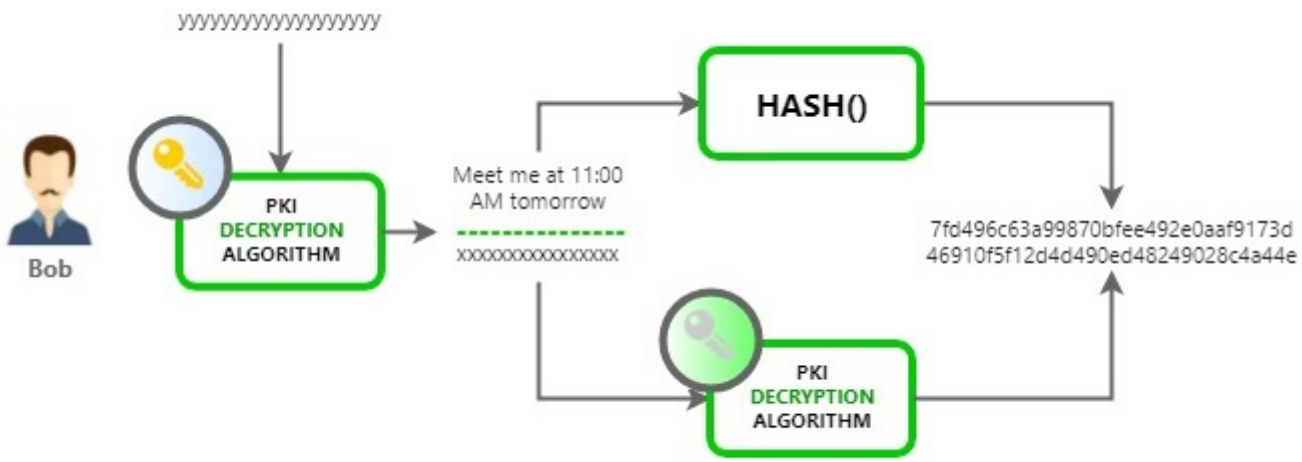


Alice sending digitally singed and encrypted message to Bob

Verifying a Digital Signature

1. Bob decrypts the message using his private key
2. Bob decrypts the digest using Alice's' public key
3. Bob computes digest of the message. If it matches the digest he received as signatures it confirms him that:

- Message is not tampered with
- Message has been sent by Alice only - Since only Alice's public key could decrypt the hash



Bob verifying the message he received from Alice

So far...

We have seen how awesome public key cryptography is and how, when combined with hash functions, it provides us with the capability to digitally sign any data.

- It helps us communicate securely on insecure channel (open internet), ensuring only the intended receiver of a message can read the message
- Ensuring the receiver can trust that the message is coming from a specific sender
- Ensuring the message is not tampered with during transmission

For all of this to work Bob needs Alice's public key. However, can he trust the public key he received indeed belongs to Alice? He received the key over the same untrusted network.

1

Which of the following is ensured by using the above method to send messages securely?

COMPLETED 0%

1 of 2

To answer the questions raised in this lesson, we need to understand the concept of PKI. Lets move on to next lesson for that.

Public Key Infrastructure

In this lesson, we will understand the concept of public key infrastructure and see how certificate authority works.

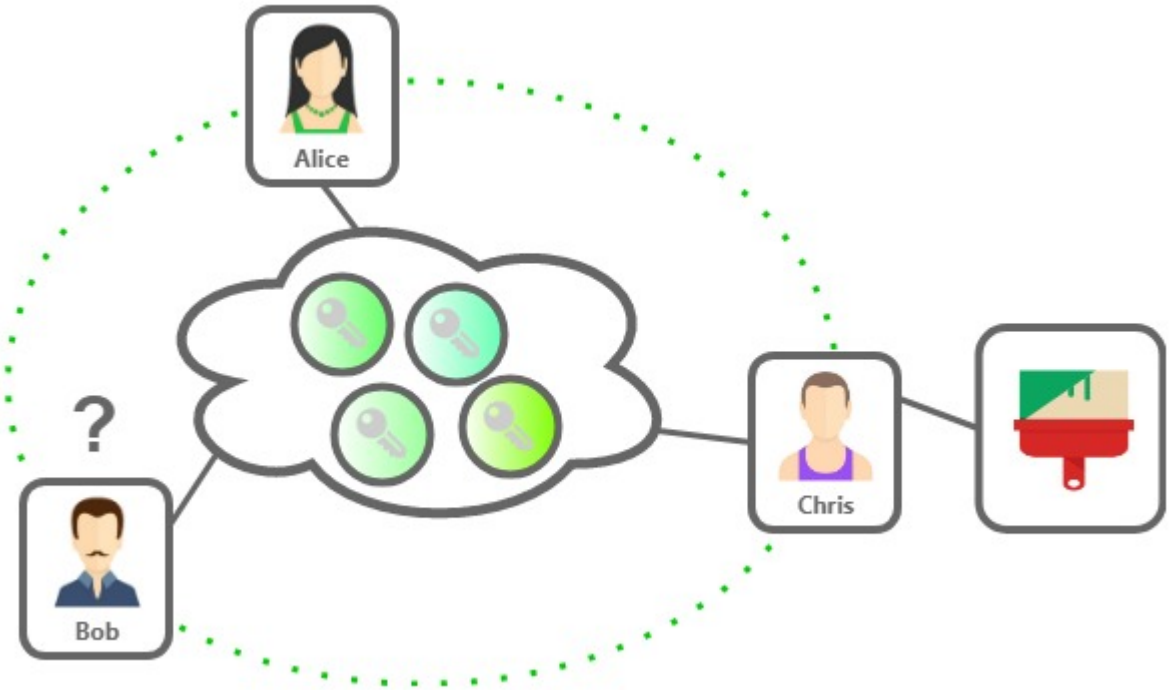
WE'LL COVER THE FOLLOWING

- Here is how it works with a CA:
- Chain of Trust

We have seen how Hash functions and asymmetric encryption can help us digitally sign any message.

On an open or insecure internet, in order for a receiver(Bob) to verify a signature done by a sender(Alice), he needs Alice's public key beforehand.

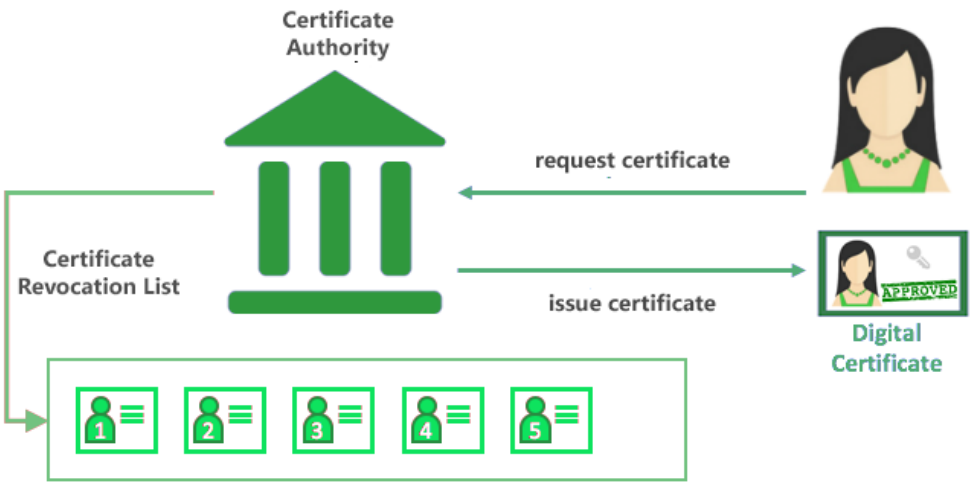
If Alice was to send her public key to Bob on the internet, how can Bob be sure that it is really Alice who has sent him her key and not Chris pretending to be Alice?



For the entire secure communication system to work, both parties(Bob and

Alice) need to establish trust in each other's public key while using the same open unsafe internet.
How to share the key and prove its ownership to ensure future secure communication?

To help solve this issue we have a concept of a trusted authority in the middle that all network participants trust. This trusted authority is called a CA or certificate authority.



Here is how it works with a CA:

1. Alice must contact the CA to get a certificate(digitally signed public key) to prove her public key to others. All other participants on the network will be able to trust the public key is Alice's if they trust the CA.

2. The CA would take Alice through an approval/on-boarding/manual-verification process and issue a certificate. The certificate itself is a list of certified attributes of the entity its issued to Alice. It has attributes like the public key, name of the holder etc. All this data is digitally signed by the CA using its own private key.

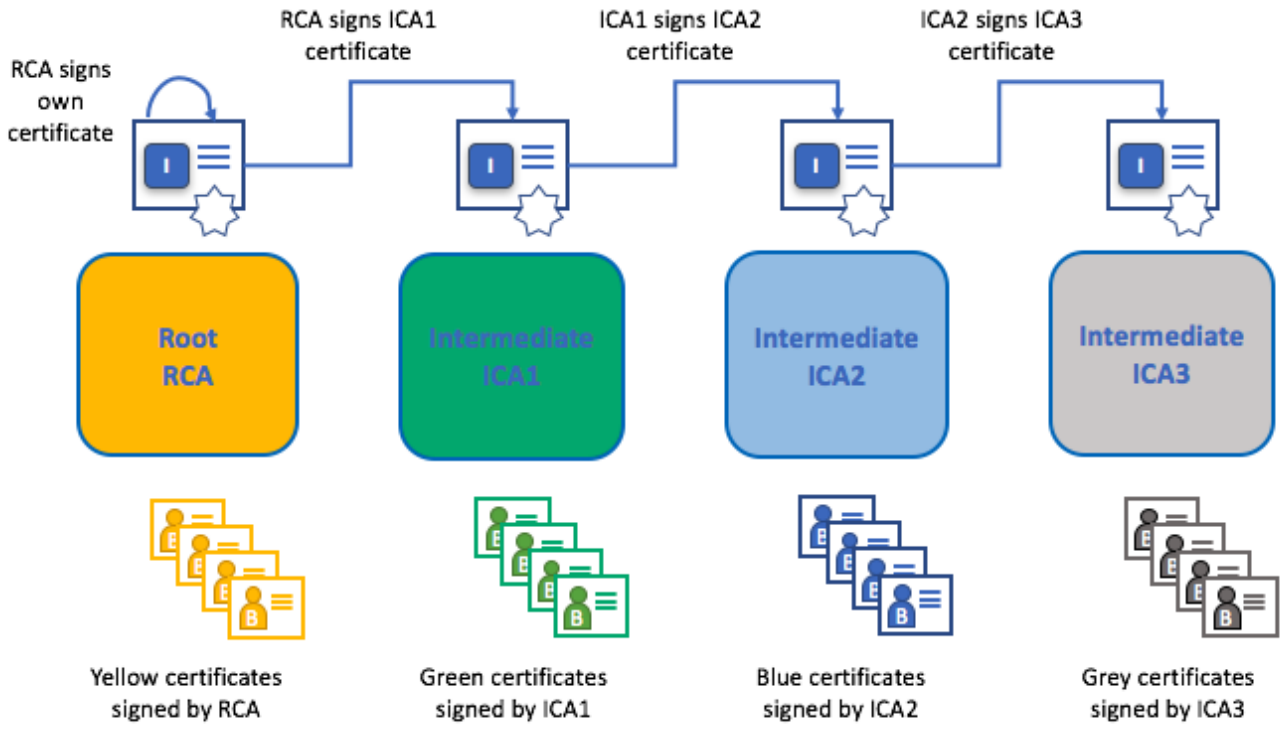
3. Alice then shares this certificate as a proof of her public key to Bob.

4. Since Bob has the CA in his trusted CA list(CA's public key), he can verify and trust the certificate shared by Alice and hence trust her public key.

Chain of Trust

A chain of trust is established between a Root CA and a set of Intermediate

CAs as long as the issuing CA for the certificate of each of these Intermediate CAs is either the Root CA itself or has a chain of trust to the Root CA.



Test Yourself

1 What is the primary purpose of a certificate authority?

COMPLETED 0%

1 of 2

In the next chapter, we will discuss blockchain data storage.

Blockchain Data Structure

In this lesson, we well discuss the data structure of blockchain

WE'LL COVER THE FOLLOWING

- What is blockchain?
- Distributed Consensus
- Data Storage in Blocks
- Activity:

What is blockchain?

In a blockchain system, data is stored in blocks of transactions. The most common definition is as follows:

> A timestamped log of transactions that is replicated on multiple peers.

Distributed Consensus

Now, some of these peers might be evil and intentionally report a tampered version of blockchain data. The entire network uses democracy to come to a consensus on the current state of data and any non-conformant or outlier peers are ignored/blocked.

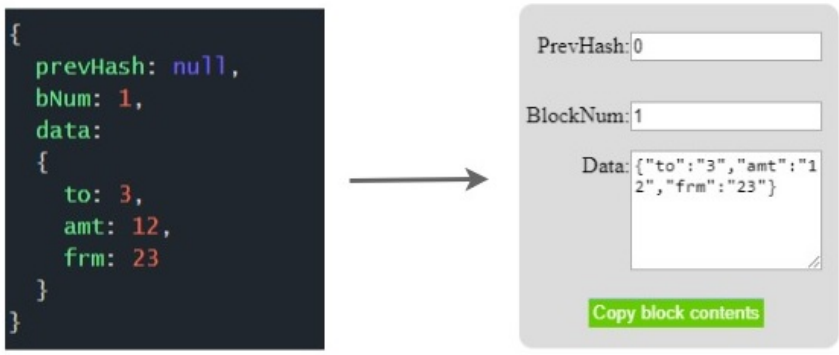
This means that in order for a blockchain network to be fair and valid, most of the nodes have to be fair. If 51% of the nodes are compromised the network is hacked. Since the networks are globally distributed, this is not a possibility.

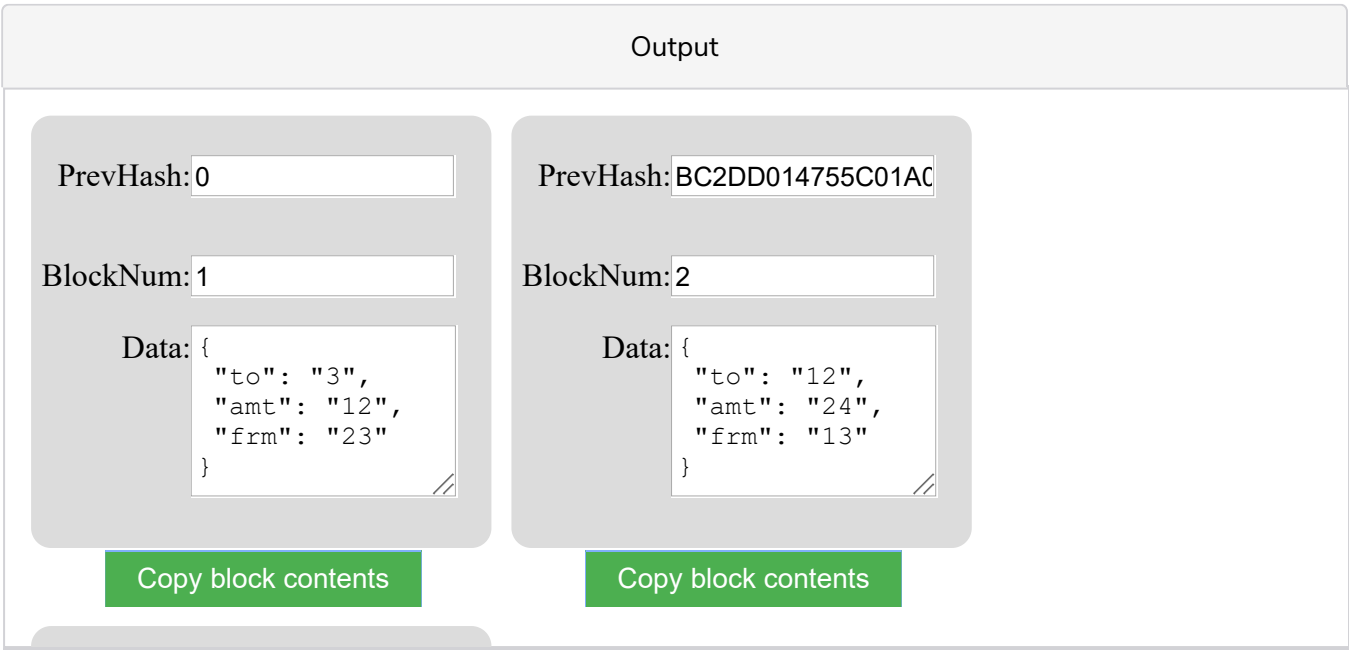
Data Storage in Blocks

Let’s see how blockchain stores data in blocks. Each block stores a data

blob(which is usually a list of transactions), its block number, and hash of the previous block:

Each block is represented by a json object here for better understanding:





Let's look at the previous block hash, referenced as "prevHash" in the above diagram.

When creating a new block, the hash of the previous block is calculated and added to the next block. Now if the previous block is altered later in time, the next block will be invalid as the prevHash stored in it will not match the actual hash of previous block.

Using one-way hash functions, the data in the blockchain is safeguarded from tampering.

Activity:

Imagine that there are three more transactions to be recorded and a new block needs to be added to the block data structure shown above. What should be the value of PrevHash in new BlockNum 4.

Use the hashing widget below to help calculate your answer.

Check Answer

View Hint

Output

HASH()

e3b0c44298fc1c149afbf4c8996fb924
7ae41e4649b934ca495991b7852b855

Test Yourself

1 What does each block in a blockchain represent?

COMPLETED 0%

1 of 2

In the next lesson, we will see the blockchain playground.

Blockchain Playground

In this lesson, we will provide you with an interactive playground in order to experiment with the Blockchain data structure.

Here is an interactive playground for you to experiment with.

Each block has the fields **data, prevHash and block number**. Try to change the data in an older block and see what happens.

PrevHash: 0

BlockNum: 1

Data:
```
{
 "to": "3",
 "amt": "12",
 "frm": "23"
}
```

Copy block contents

PrevHash: BC2DD014755C01A0

BlockNum: 2

Data:
```
{
 "to": "12",
 "amt": "24",
 "frm": "13"
}
```

Copy block contents

PrevHash: B8A28D54CAE5429A

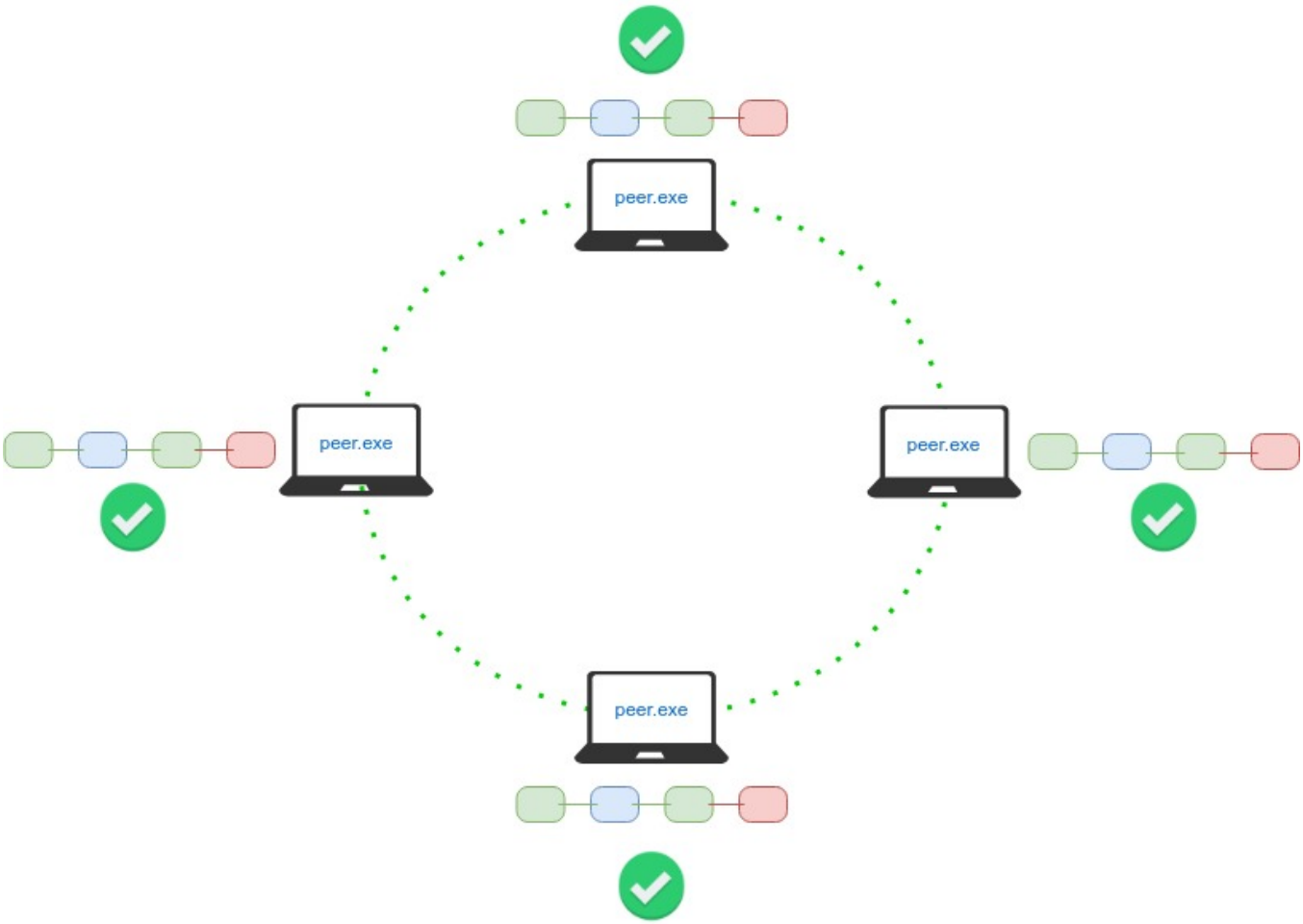
In the next lesson, we will see the bigger picture of Blockchain.

Blockchain Big Picture

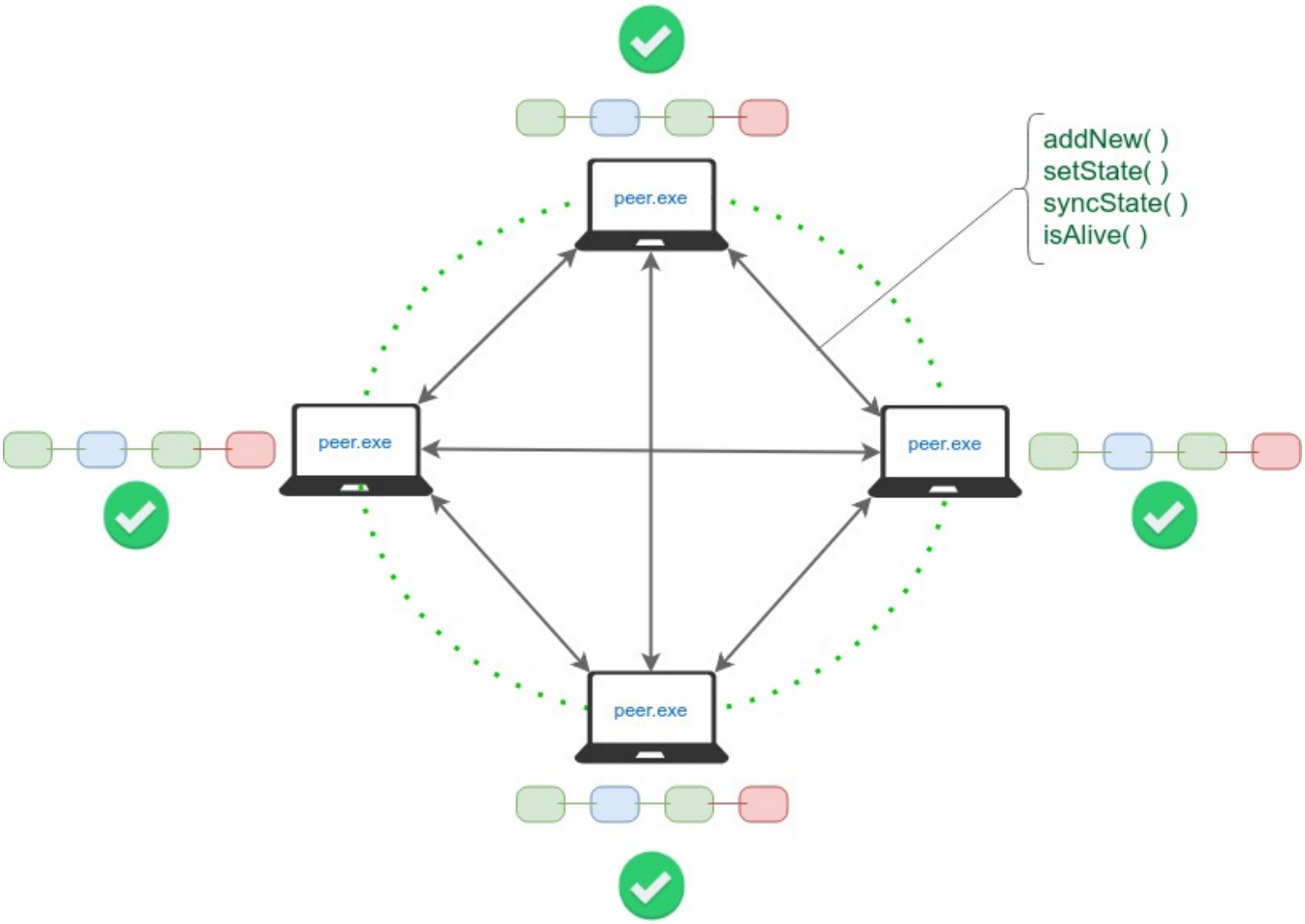
In this lesson, we will see the bigger picture of Blockchain.

You have seen how we can store data in a structure called blockchain. In a distributed blockchain network:

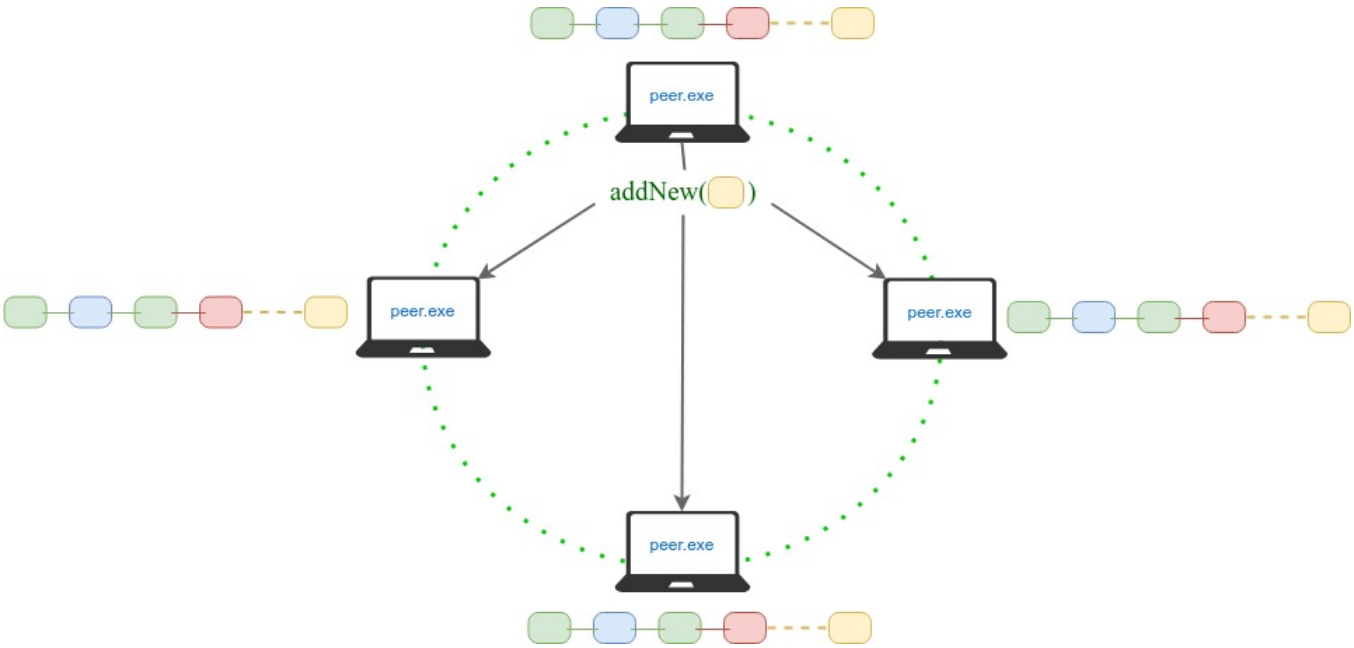
Multiple peers have a process(peer.exe) running that maintains a ledger on their local storage.



This process connects to other process instances running on other machines to receive updates on new blocks, transactions, health and validity checks etc to keep itself updated.



A transaction/block can be appended by any peer and is then broadcast-ed to all peers.



Since multiple peers are adding transactions/blocks simultaneously, the consensus protocol along with the ledger implementation ensures “validity” and ordering of transactions in blocks forming the blockchain.

We will dig in to the consensus and validity more in future lessons. Let’s start with the Bitcoin example to see how all this can create real value.

In the next chapter, we will understand the value of Blockchain through the example of Bitcoin.

The Bitcoin Blockchain History

In this lesson, we will check out the bitcoin history and how transactions work in bitcoins.

WE'LL COVER THE FOLLOWING

- Membership/Transaction Signing:
- Calculating Account Totals:
- Coin Generation:

The Bitcoin blockchain was born in 2009, by a pseudonym Satoshi Nakomoto.

Take a look here at the first block(block 0) or the "genesis block" of Bitcoin blockchain.

There is one transaction in this block to Satoshi Nakomoto's address. Interestingly Satoshi's account address has never been used to transfer out money so we don't know if it's intentional, or if the private key lost?

Now look at another block, this one has 1660 transactions.

> A transaction is a record of transfer of bitcoins from one address to another.

Membership/Transaction Signing:

A transaction is always digitally signed by the 'from' an account holder to ensure that only the owner of account can perform a transaction using their balance.

In Bitcoin, anyone can generate an asymmetric keypair and start transacting on the Bitcoin blockchain.

```
{
    to:   ...   ,
    from: ...   ,
    amount: ...
}
```

Calculating Account Totals:

Consider three blocks with multiple transactions in each block. The timestamp has been omitted here to give a simplified view:

Block No: 1	**Block No:** 2	**Block No:** 3
Prev Hash: 0	**Prev Hash:** xxxxxxxxxxxxx	**Prev Hash:** xxxxxxxxxxxxx
Data:	**Data:**	**Data:**
- To: A, Frm: Sys, Amount: 100	- To: D, Frm: Sys, Amount: 100	- To: C, Frm: Sys, Amount: 100
- To: B, Frm: A, Amount: 50	- To: B, Frm: A, Amount: 20	- To: D, Frm: C, Amount: 50
- To: C, Frm: A, Amount: 10		

This would mean the accounts totals for each accounts are as follows, after executing all transactions:

- A: 20
- B: 70
- C: 60
- D: 150

Coin Generation:

In our example, each block has a transaction from “Sys” which awards some balance to an account. In Bitcoin, this is labelled as “newly generated coins” and these are the new coins the system awards to the miner of the block.

Now lets see what *mining* is...

Test Yourself

1

Each block in Bitcoin has a fixed number of transactions.

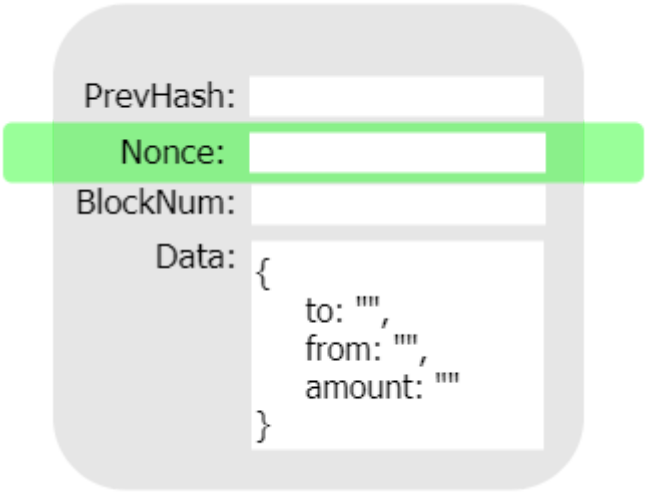
Bitcoin Mining

WE'LL COVER THE FOLLOWING

- Mining
- Nonce Mining
- Exercise - Mining

Mining

In Bitcoin implementation there is an extra field added in each block, called `nonce`.



Mining peers continuously receive new transactions submitted on the chain. In order for a transaction to be confirmed, it should be put in a block, that needs to be created.

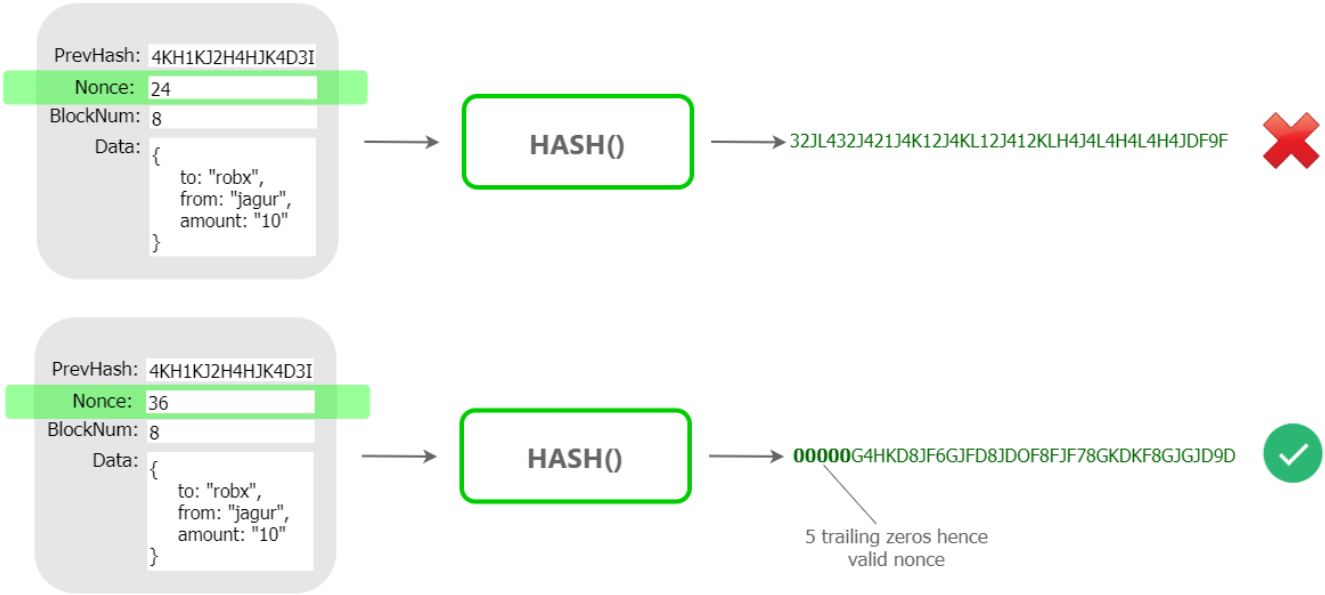
Simply adding transactions to a block is simple - you get the hash or prev block, increment the block number, add the list of transactions, and take its hash. Pretty fast! But in order to incentivize to run a node, Bitcoin awards miners for creating new blocks by giving them newly generated Bitcoins.

Since it would be very simple to mine a block and devalue the generated coin,

Bitcoin has made the block creation process challenging. This difficulty self-adjusts to ensure the global pool of miners are able to mine around one block every 10 seconds. This ensures that the supply of new coins is not too fast, making bitcoin a valuable asset.

Nonce Mining

So Bitcoin adds this extra requirement for a block to have a `nonce` value, such that the block hash of the new block with the nonce has X number of trailing zeros. This X is adjusted by the protocol to control the speed of coin generation. The higher the value of X, the harder it is to calculate a nonce.



Now since hash functions are one-way functions the nonce cannot really be 'calculated'. Miners do a hit and trial kind of algorithm to see if they can find a nonce that satisfies the requirement(X trailing zeros) for the next block.

If we go back to our block # 40000 that we looked at earlier, you will find that its hash is `0000000000000000004ec466ce4732fe6f1ed1cddc2ed4b328fff5224276e3f6f` which has 17 trailing zeros.This means that when this block was mined in 2016, the value for X was 17.

Exercise - Mining

The purpose of this exercise is to understand how block mining is 'hard'.

Let's say you have a list of transactions that you are mining a block for. And you are required to find a value for nonce that creates blockhash with 3 trailing zeros. Input a value of nonce and the hash will adjust. See if you can find a good nonce in a minute.

find a good nonce in a minute.

PrevHash: 0

Nonce:

BlockNum: 32

Data:
```
{
 "to": "robx",
 "from":
"jagur",
 "amount": "10"
```

HASH()

Check Answer View Hint

Test Yourself

Q Mining a bitcoin essentialy is:

COMPLETED 0%

1 of 1

In the next lesson, we will discuss the value of bitcoin...

How Bitcoin Becomes Valuable

In this lesson, we will discuss how Bitcoin became a recognized currency.

Value for anything is driven by the balance of its demand and supply.

Anything found in abundance is of no great value. So, in order for something to be valuable, it must be limited in supply. Bitcoin is limited due to the proof-of-work requirement that adds new Bitcoins into the system.

Demand is higher for things people find useful. It could be something useful for two reasons:

1. In terms of the actual utility it provides.

- For example, cars provide a transportation utility.
- Bitcoin provides instant value transfer facilities which is not found in traditional currencies.

2. In terms of the satisfaction it brings.

- For example, people like to have diamonds for physiological satisfaction. Crystals are just as comparable in terms of utility.
- Decoration pieces - aesthetic satisfaction.

A supply-demand balance makes Bitcoin a valuable and desirable currency.

It's nothing physical. It's just a digital, immutable, trustable log of transfer transactions. But all these properties give it great utility and therefore value!

It's important to keep in mind that money doesn't really exist. 'Moneyness' is a property. This property of moneyness is held by fiat currency (legal tender whose value is backed by the government that issued it)and bills due to the State Bank (trusted authority) that promises to hold its value. This 'moneyness' is in anything that we deem valuable and therefore can be used as a unit of account, a store of value, or a medium of exchange.

Bitcoin has these properties of ‘moneyness’ making it a valuable asset.

- It allows you to store value
- It allows you to consume value (better than fiat)
- It allows you to exchange value (better than fiat)

In the next chapter, we will discuss the concepts of HyperLedger Fabric.

Introduction

In this lesson, we will give a quick introduction to Hyperledger Fabric.

Blockchain technology is also referred to as Distributed Ledger Technology, or DLT, as it works on a ledger that is distributed on multiple peers.

Now that we have a high-level understanding of what DLT is we are ready to jump into Hyperledger Fabric.

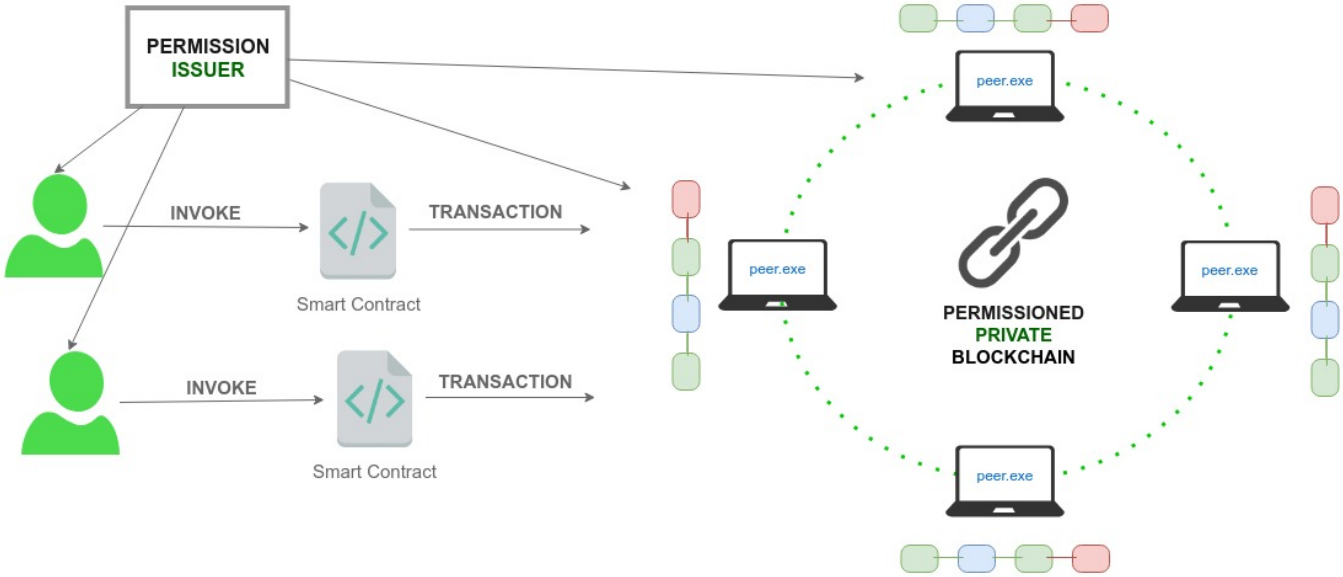
In this section, we will understand core Hyperledger fabric concepts and terminology at a high level and explore a “hello world” smart contract built on Hyperledger fabric.

Hyperledger Fabric is a “blockchain platform for the enterprise”. It is open source and modular - allowing different modules to be used, plug and play style. This enables a wide variety of enterprise requirements. It is designed to provide speed and scalability that is lacking in public chains due to *proof of work* requirement, which is essentially nuance mining.

Hyperledger fabric is ideal for building a permissioned, private blockchain business network. By private, it means that it should not be publicly open for everyone to run a peer or transact on the network. For enterprises, this a big requirement that Hyperledger fabric meets. Enterprises need more control on their data access policies. They also need a permissioned network so they can implement access control as per their own requirements.

At a high level, this is how a Hyperledger network works. The permission issuer issues or revokes permissions for all participants and infrastructure components of the network. This permission or access control in Fabric is based on X509 PKI infrastructure. Which means there is a trusted certificate authority that issues certificates to all participants.

Smart contracts hold logic that defines who can change what on the ledger. And participants write transactions on the ledger by invoking smart contracts.



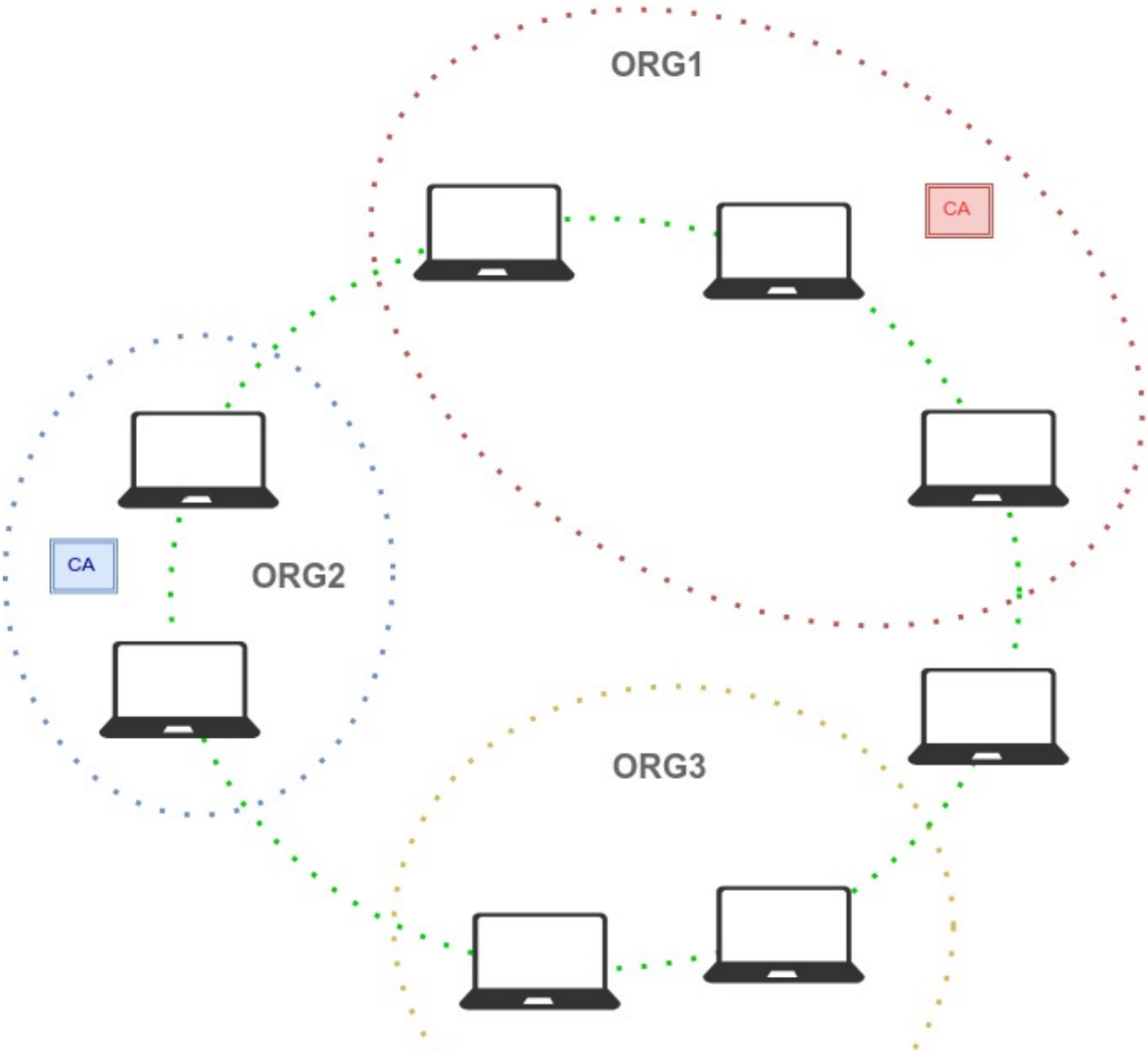
In the next lesson we will understand hyperledger network concepts

Network Concepts

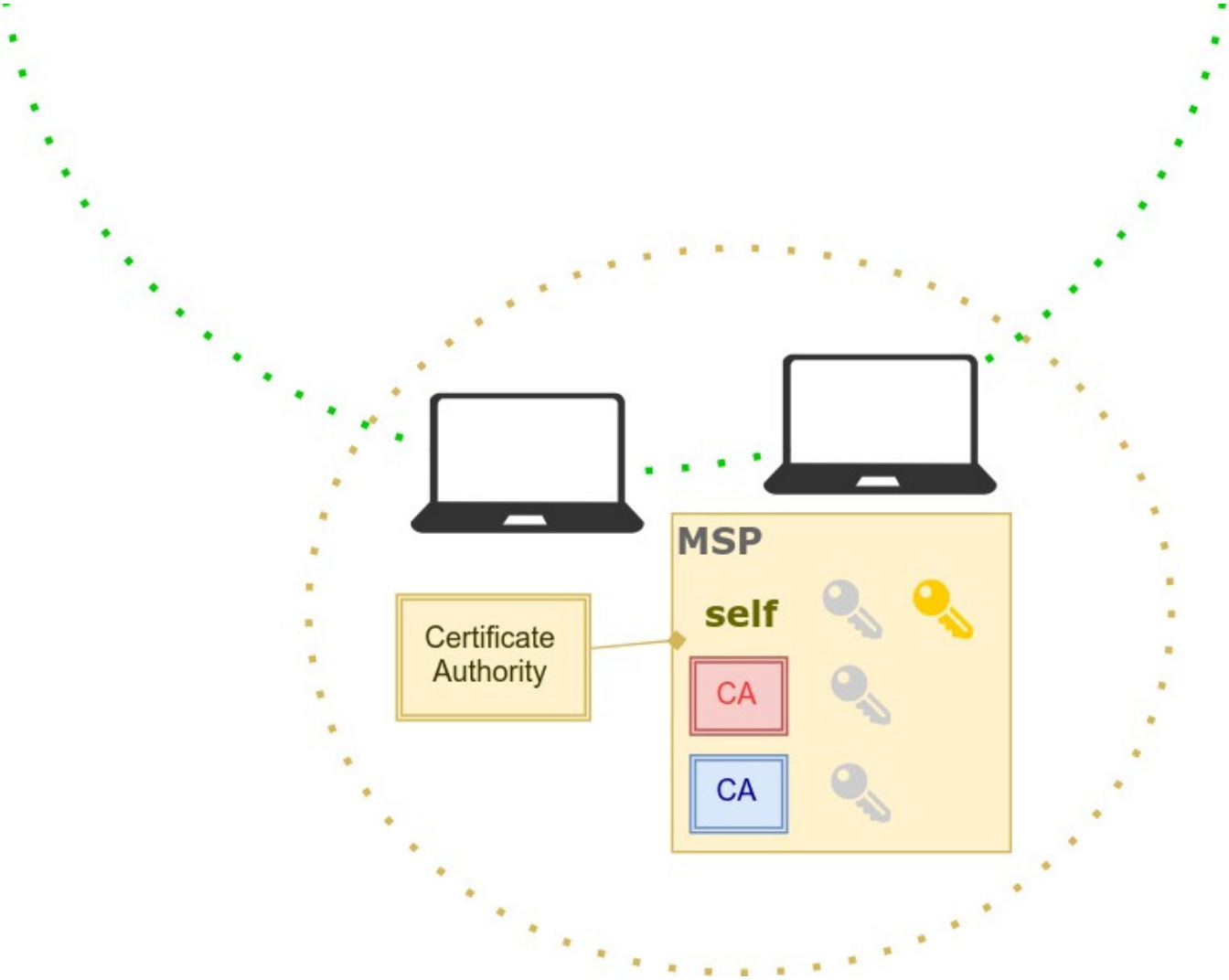
In this lesson, we will give a quick introduction to Hyperledger Fabric.

WE'LL COVER THE FOLLOWING

- Channels
  - Ledger
  - Chaincode
  - Ordering Service



Multiple organizations can form a network. Each organization can have its own membership services provider which will issue and revoke identities for users and peers of that organization. This membership service provider can be a certificate authority hierarchy or a single root certificate authority. In order for other organizations to validate transactions by each other, they need to be set up by having trusted root certificate authorities pre-configured on them in something called an MSP (Membership Service Provider).

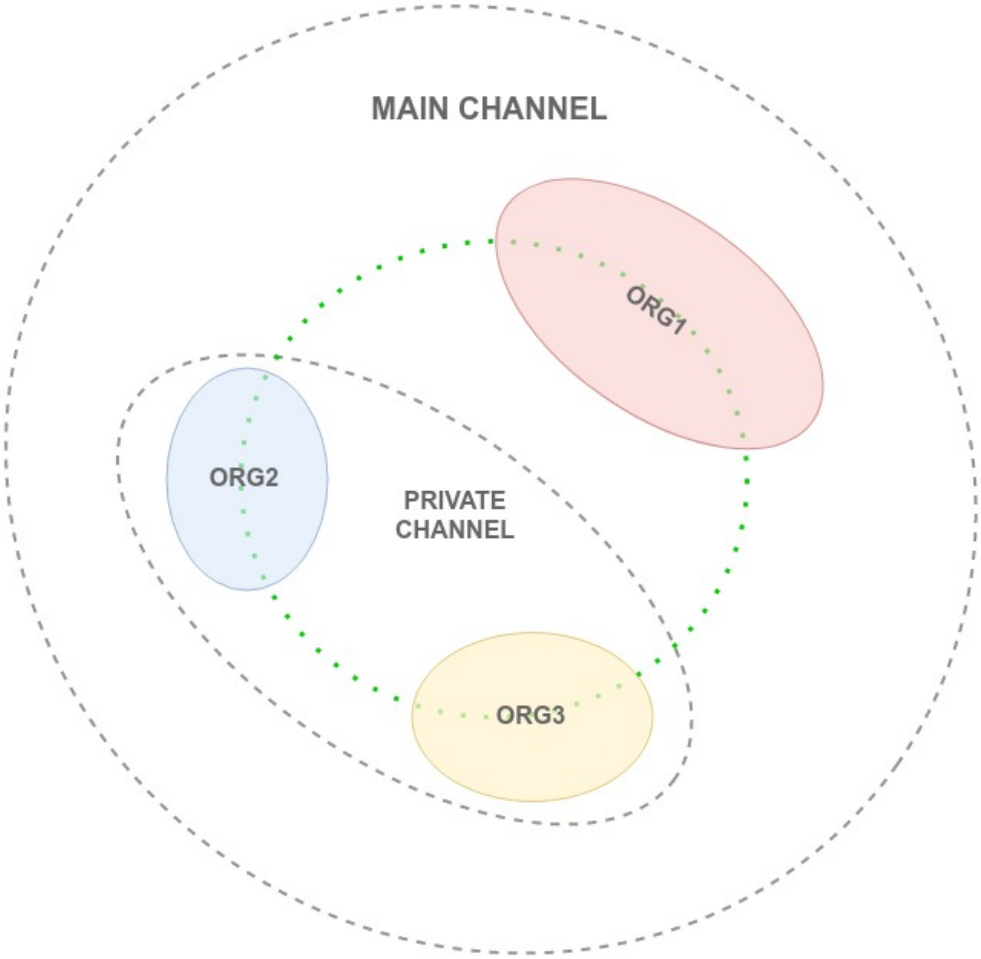


Channels

In fabric there is a concept of channels. Each peer can join one or more channels. If there are some special transactions that need to be shared only by org1 and org2 and they don't want any peer from org3 to read it, both the organizations can join a separate channel and be present on the main channel as well.

This provides privacy that does not exist in public blockchains.

Internally a separate ledger is maintained on each peer for each channel that it is on.



Ledger

Each peer maintains a ledger. A ledger is essentially two things:

- Timestamped transactions organized in blocks and stored.
- A State Database which is the computed outcome of all transactions executed in order.

By having a state database it is easier to query the ledger as the current state is pre-calculated and stored on each peer.

The **state database** is essentially a key value based datastore(an instance of Apache CouchDB). The state database contains the final state of the ledger after applying all transactions recorded in blockchain. It is therefore like a cache. It helps make querying the blockchain faster, as the state is pre-calculated. It also gives developers an easier programming model to work with. Developers writing chaincode do not need to write transactions directly on blockchain, but rather, write to state in state database (the required transactions are generated under the hood).

The peers communicate with each other to ensure their final state is the same at all times using the *gossip protocol*.

Chaincode

This is a piece of code that is part of the ledger. The chaincode provides logic on what, how, when, and by whom things can be written on ledger. It is essentially business rules coded to store data onto the ledger. For example the chaincode can make sure an account holder has enough balance before he transfers an amount to another account.

Ordering Service

Ordering Service is a distributed(solo in dev mode) service that is responsible for organizing endorsed transactions into sequenced blocks and distributing to all peers. This service is run on multiple nodes. Typically each org will have at least one node of ordering service to ensure they are a part of the end to end transaction processing.

In the next lesson, we will take a look at Transaction processing.

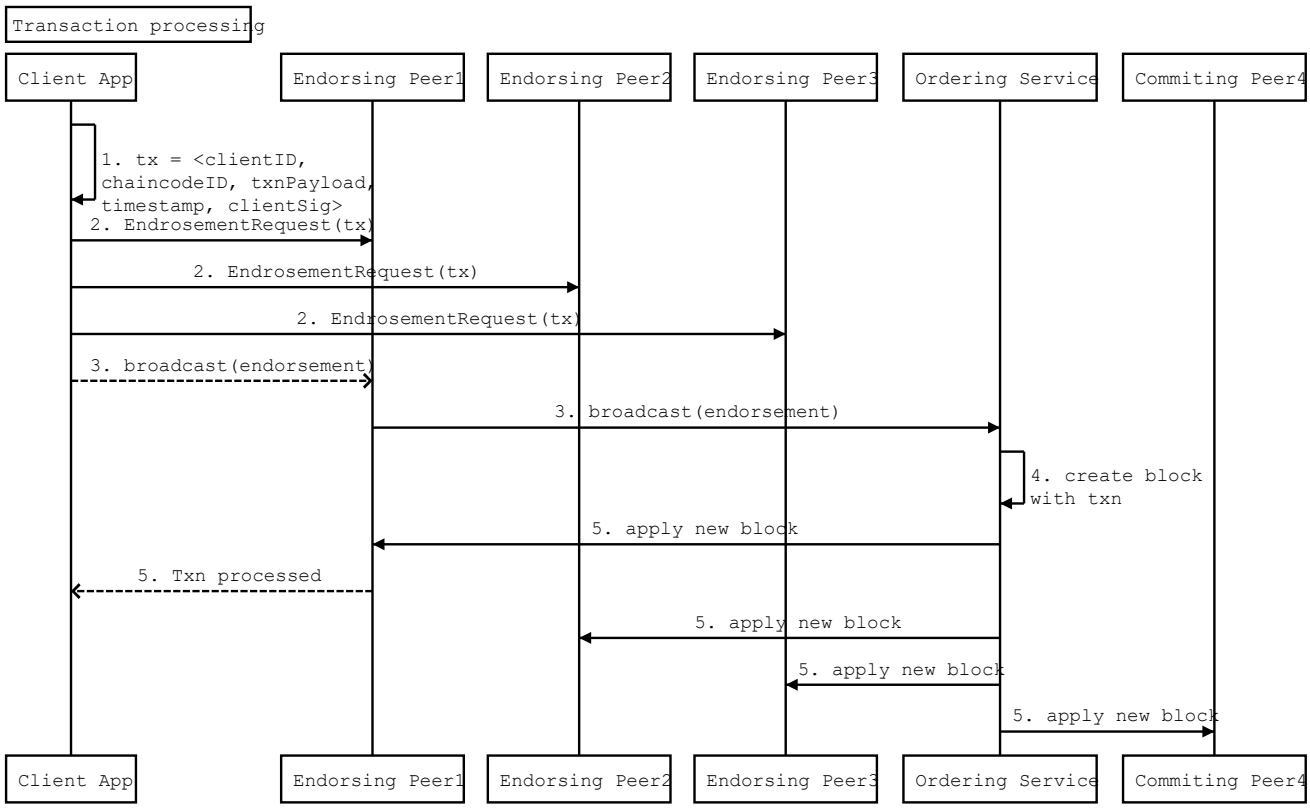
Transaction Processing

In this lesson, we will take a look at high-level steps carried out in transaction processing.

Here is the list of high-level steps being carried out when a transaction needs to be applied to the ledger.

1. Client App prepares the transaction signed by the user.
2. Client App connects to endorsing peers(as per endorsement policy) to collected signed endorsements on the transaction output. The peers simulate the transaction and return a signed endorsement.
3. Client App submits the endorsed transaction to ordering service, which puts it into a valid block and distributes to all peers.

Please go through the detailed flow using this diagram:



In the next lesson, we will take a look at development components.

Development Components

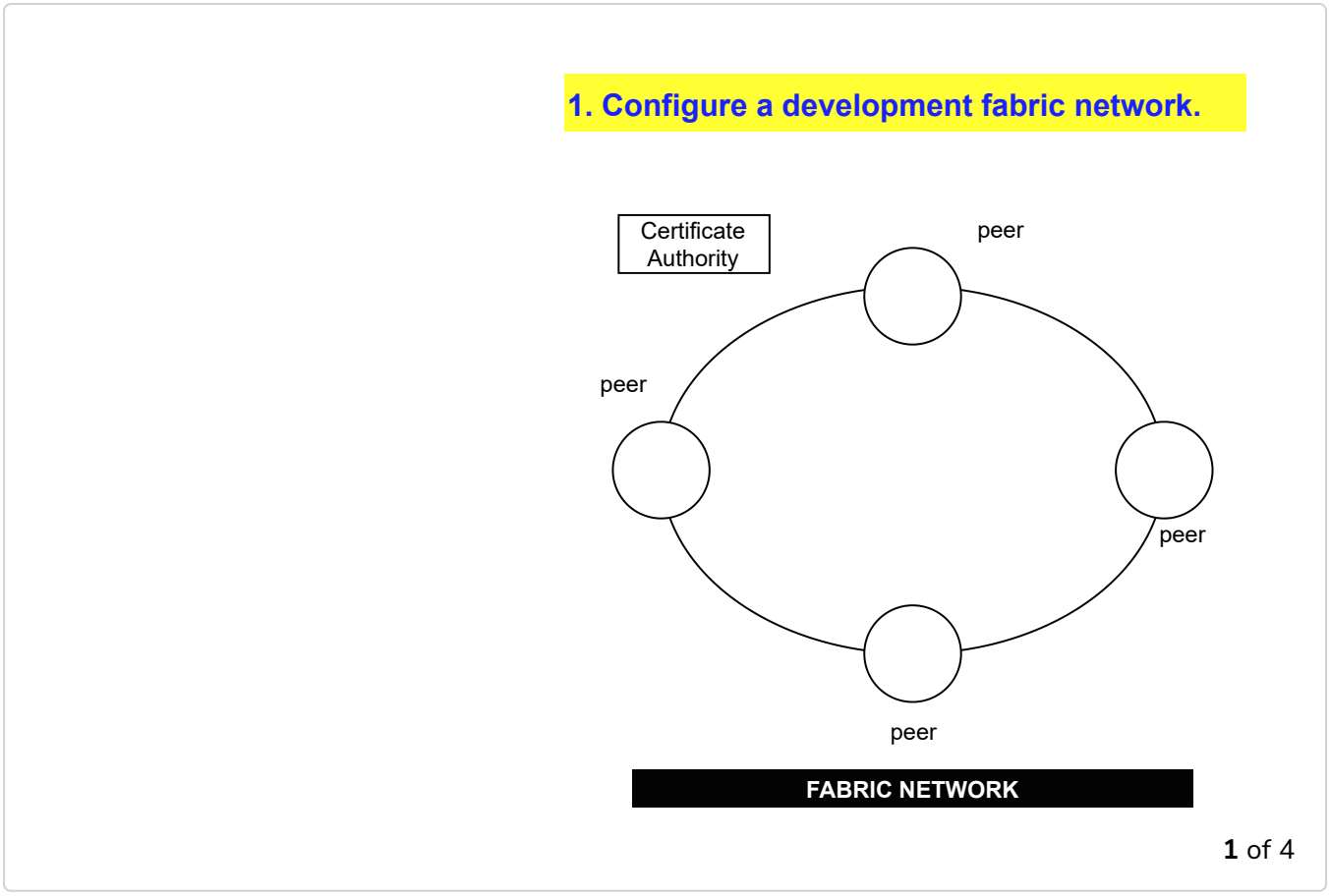
In this lesson, we will take a look at how components are used to deploy a network.

WE'LL COVER THE FOLLOWING

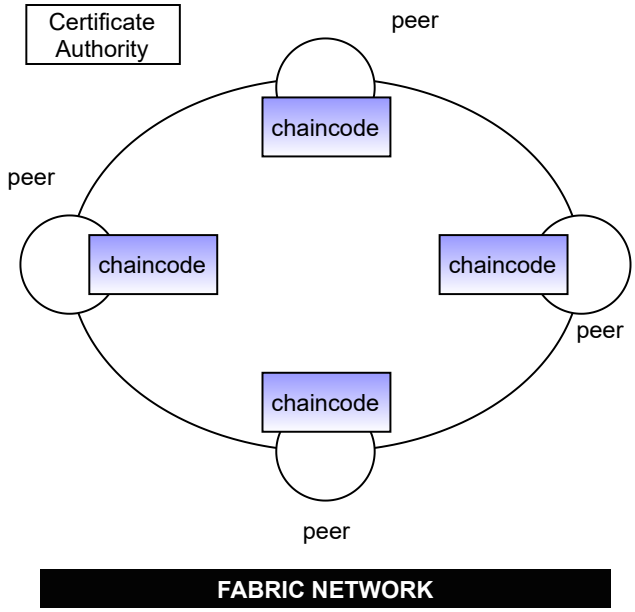
- Deploying a network

There are 3 main development components for a basic end-to-end development flow:

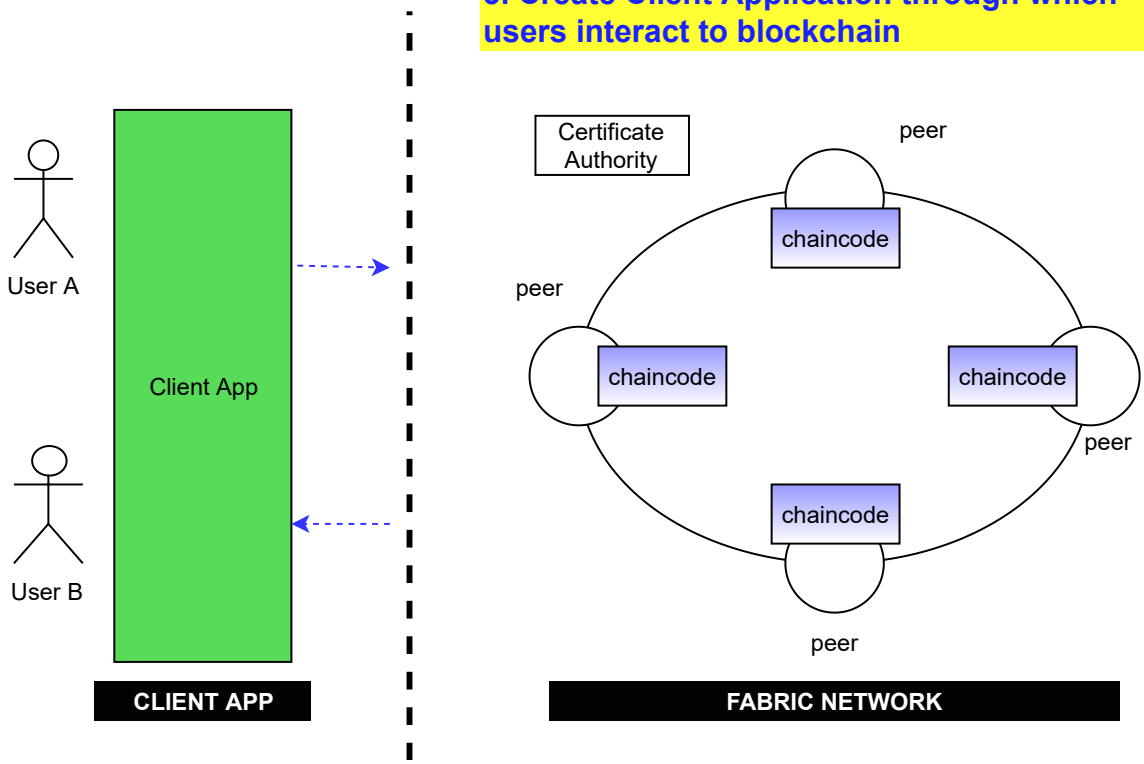
1. Configure a `dev network`
2. Write and deploy `chaincode` on the `dev network`
3. Write application code that can invoke chaincode transactions on dev network.

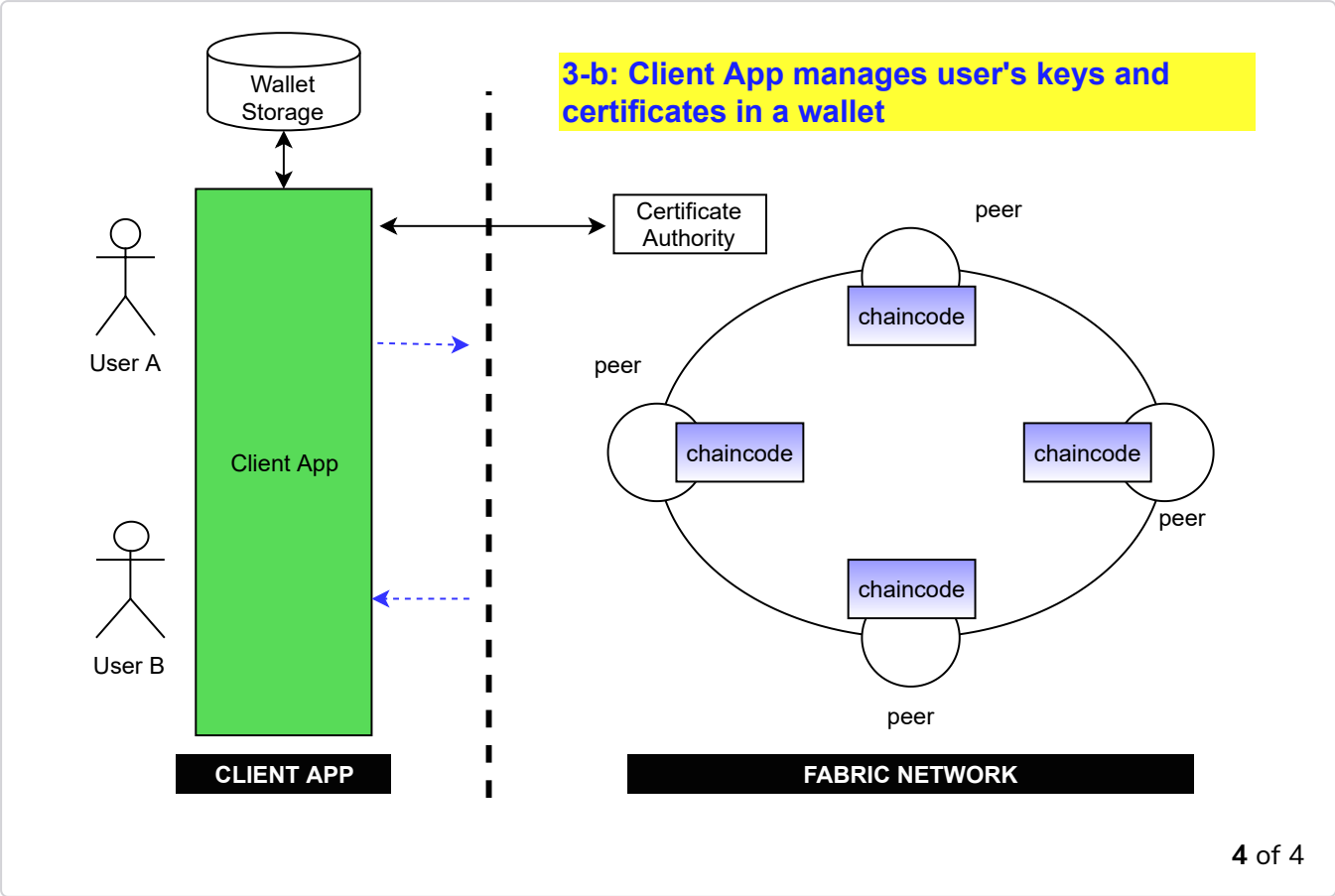


2. Write and deploy chaincode



3. Create Client Application through which users interact to blockchain





Deploying a network

Typically a network architect and network operator will deploy a network which will be distributed on multiple machines that are connected.

For development purposes however, we can launch a simple network on our developer's local machine so we can test our code on it.

The developer would first:

1. Deploy a basic network on their local machine
2. Write chaincode. Deploy their chaincode on his basic network
3. Write application code that invokes the chaincode they deployed.

In the next chapter, we will deploy our very own Fabric network.

Network Concepts

We will see how we can deploy a basic hyperledger fabric network to enable local development.

WE'LL COVER THE FOLLOWING

- Understanding Docker
- Components of Network:
  - PEER:
  - ORDERER:
  - CERTIFICATE AUTHORITY:
  - TOOLING:
- Pre-reqs to network up:

Hyperledger Fabric forms a distributed network and in order to run on a single machine, we use docker containers to run individual distributed components. Each component runs in a separate container instance and connects to other containers to form a network.

Understanding Docker

A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings. Understanding docker and docker-compose is crucial to understanding the dev environment we will use in this course.

Please visit the following links to get a basic knowledge about docker if you do not already have it:

- What Is Docker And How It Works?
- Make a Local Docker Container for php

For development purposes we will run a small network on our machine in

For development purposes we will run a small network on our machine in docker containers. This network configuration is called basic-network and is

provided in official hyperledger fabric-samples repo (link: https://github.com/hyperledger/fabric-samples).

Components of Network:

Lets get to know the components of a basic network.

PEER:

- **peer container**: Runs the peer node
- **couch db container**: Stores the state database of peer node

ORDERER:

- **orderer container**: Solo orderer node to keep dev environment simple. In a real-world network the ordering service is distributed with multiple nodes communicating with each other.

CERTIFICATE AUTHORITY:

- **fabric-ca container**: root certificate authority for issuing membership certificates to all nodes and users. Since we will use single CA, our network is comprised of a single org called “example dot com”.

TOOLING:

- **fabric-cli container**: This container has some cli tools that help us interact with network nodes to deploy chaincode etc.

Pre-reqs to network up:

Before we can bring up a network we need to do a couple of things:

1. Generate keys and certificates for all infrastructure entities. The key will be given to each entity and the certificate with the public key will be provided to all other entities. Each certificate generated will be signed by common `certificate-authority` or `CA`. There are multiple ways of generating this essential crypto material. To keep things simple for now, we will use pre-generated crypto material `/infra-basic-network/crypto-config`. This material was generated using the `cryptogen` utility provided by the x`xhyperledger fabric. We could also use OpenSSL to generate

this.

2. Generate `genesis block` and `config transaction`. You will do this using `confixtxgen` tool.

Now lets deploy our first network.

In the next lesson, we will deploy our network.

Deploy Your Network

In this lesson, we will launch an environment on which we will run all our network containers(peer, orderer, CA, state-db)

WE'LL COVER THE FOLLOWING

- Folder Structure:
- Execution Environment for STEP 1:
- Steps to Run the Environment:

Folder Structure:

First lets take a look at the folder structure. Here is what's inside the `infra-basic-network` folder:

- **Crypto Config**:
  - `crypto-config.yaml` : This is the input file to `cryptogen` tool for generating the crypto material. This tool has already been run for convenience in our environment and resultant keys and certificates are already added in `crypto-config` folder.
- **Configuration Transaction**:
  - `configtx.yaml` : config file defining input to generate the configuration transaction. This will be consumed by `configtx` executable to generate config transaction in files inside infra-basic-network/config folder.
- **Network Components Containers Definition**:
  - `docker-compose.yml` : This file defines each infrastructure component that will run to form the network. If you look inside this file you will see it defines all the containers using official Hyperledger Fabric

images. The crypto-config folder is mounted as volume on these containers to give them access to relevant keys & certs.

Execution Environment for STEP 1:

```
-----BEGIN PRIVATE KEY-----
MIGHAgEAMBMGByqGSM49AgEGCCqGSM49AwEHBG0wawIBAQQgbIRfBJjk/t3HLnEz
32V4sCllmJtnliVv4UmLfrjZ+B6hRANCAASM01iiFoDgTsTd27nU+R1z7YZbqM4I
Tlz13Mg+SQWsWn25IM6/IwtzNq5SSQZtJwpo7+gtS5IggDn7WJMi6Hy6
-----END PRIVATE KEY-----
```

Steps to Run the Environment:

1. Press the RUN button. Wait for environment to be up and the terminal to show up. Ignore the URL being shown for accessing the app since we will stay in the terminal for now.
   In the usercode directory all the code you see in the widget is mounted. Run the following command as we will run all commands in the infra directory:

```
cd /usercode/infra-basic-network
```

Now we will generate the genesis block and configuration transaction as they are needed before we can bring up the network. Genesis block is the first block in the chain, which contains the channel's initial configuration.

2. Generate genesis block for orderer using the `configtxgen` tool which is already on this environment.

```
/workspace/bin/configtxgen -profile OneOrgOrdererGenesis \
                           -outputBlock ./config/genesis.block
```

This command will generate the `genesis.block` file in config folder using the `configtx.yaml` file as input. The genesis.block file is accessed by the orderer node defined in docker-compose.yaml through a mounted volume.

3. Generate channel configuration transaction

```
/workspace/bin/configtxgen -profile OneOrgChannel \
                           -outputCreateChannelTx ./config/channel.tx \
                           -channelID mychannel
```

4. Generate anchor peer transaction

```
/workspace/bin/configtxgen -profile OneOrgChannel \
                           -outputAnchorPeersUpdate ./config/Org1MSPanchor
s.tx \
                           -channelID mychannel \
                           -asOrg Org1MSP
```

5. Now we will bring up the network using `docker-compose up` command. If you do not have knowledge of `docker` and `docker-compose` it is recommended that you take some tutorials so the docker-compose.yaml file starts to make sense for you. Run the following to launch your containers. On first run, it will download fabric official images from docker hub - be patient as this is one time per environment. If you see an error about docker network, please ignore.

```
docker-compose up -d
```

Ensure all your containers are up by running the following. A total of 5 containers should be running. One of these is the container your terminal is running in. All others are the ones you just launched with your docker-compose up command.

```
docker ps
```

Now all the components are up. But we need to create a channel and join our peer to that channel. If we had multiple peers, we would have to join all of those to this channel using the same step.

6. Create the channel

```
docker exec -e "CORE_PEER_LOCALMSPID=Org1MSP" \
            -e "CORE_PEER_MSPCONFIGPATH=/etc/hyperledger/msp/users/Admin@o
rg1.example.com/msp" \
       peer0.org1.example.com \
            peer channel create -o orderer.example.com:7050 \
                                -c mychannel \
                                -f /etc/hyperledger/configtx/channel.tx
```

This command executes a command in peer0 container. The command it runs is `peer channel create`. The peer command it again provided by official

is `peer channel create`. The peer command it again provided by official

hyperledger setup. It takes the path to channel configuration transaction(channel.tx) we generated earlier.

7. Join peer0 to the channel.

```
docker exec -e "CORE_PEER_LOCALMSPID=Org1MSP" \
            -e "CORE_PEER_MSPCONFIGPATH=/etc/hyperledger/msp/users/Admin@o
rg1.example.com/msp" \
       peer0.org1.example.com \
            peer channel join -b mychannel.block
```

If you see a success message at the end, CONGRATULATIONS! Your first fabric network is up. It has no chaincode, yet. But all network components are up and connected and a channel has been formed.

To explore further, you can view logs of any container for example try these commands:

```
docker logs ca.example.com
docker logs peer0.org1.example.com
docker logs orderer.example.com
```

Again, having a good grasp on docker is crucial in understanding what is going on here as fabric setup is heavily based on docker.

In the next chapter, we will write and deploy chaincode.

Understanding Chaincode

Lets look at our first chaincode application and deploy it to our basic-network.

WE'LL COVER THE FOLLOWING

- How chaincode works

Lets start with a simple chaincode to ensure we understand the basic concepts.

We will use a simple Land Record(landrec) chaincode which will do CRUD operations on basic land records.

We will be using javascript(node) as the language for our chaincode. GO is also supported though.

We will use the `fabric-contract-api` node package to write our chaincode.

How chaincode works

Chaincode application is simple. It stores and retrieves data from underlying world-state databases and can hold any logic. Since couch db is a key-value based db, it is quite simple. Based on the data it changes in the world-state the fabric peer itself generates the transaction with “changed” data.

The `fabric-contract-api` requires our chaincode class to extend the `Contract` class.

- The `async initLedger(ctx)` method is called whenever a chaincode is initialized.
- We can add more methods depending on what functionality we want our chaincode to expose. There are a couple of important methods passed in context object that allow us to store and retrieve data from the key value store.

Look at the sample code below. There are two functions supported by this chaincode. CreateLand and QueryLand. Once this chaincode is deployed, any application can call these methods given that it calls it signed with a valid user's private key whose certificate is issued by our CA. Important lines are highlighted

```
'use strict';

const { Contract } = require('fabric-contract-api');

class LandRec extends Contract {

    async initLedger(ctx) {
        console.info('============= START : Initialize Ledger ===========');
        // todo
        console.info('============= END : Initialize Ledger ===========');
    }

    async queryLand(ctx, landNumber) {
        const landAsBytes = await ctx.stub.getState(landNumber); // get the land from chainco
        if (!landAsBytes || landAsBytes.length === 0) {
            throw new Error(`${landNumber} does not exist`);
        }
        console.log(landAsBytes.toString());
        return landAsBytes.toString();
    }

    async createLand(ctx, landNumber, address, location, owner) {
        console.info('============= START : Create Land ===========');

        const land = {
            docType: 'land',
            address,
            location,
            owner,
        };

        await ctx.stub.putState(landNumber, Buffer.from(JSON.stringify(land)));
        console.info('============= END : Create Land ===========');
    }
}
```

For more details on this visit: https://fabric-shim.github.io/release-1.4/tutorial-deep-dive-contract-interface.html

After we have written our chaincode, we will:

1. Deploy chaincode on peer(s)
2. Instantiate chaincode on peer(s) and define an endorsement policy.

Both, 1 and 2 are done using a cli tool provided by hyperledger fabric. This cli tool is preinstalled in our fabric-cli container to make environment setup easier for us. The fabric cli container is also based off the official hyperledger fabric tools image.

Lets do this practically to understand better! In the next lesson, we will deploy our chaincode.

Deploy Your Chaincode

Lets look at our first chaincode application and deploy it to our basic-network.

WE'LL COVER THE FOLLOWING

- Folder Structure for `Chaincode` Folder
- Environment for Step 2
- Running the Environment
  - 1. Run Fabric Network
  - 2. Launch Tools Container
  - 3. Install chaincode
  - 4. Instantiate chaincode
  - 5. Test chaincode invoke

Folder Structure for `Chaincode` Folder

Please note that `infra-basic-network` has all the stuff we used earlier to bring up a network. We will use the same network now and deploy our landrec chaincode on it.

Now lets look at important files inside the `chaincode` folder.

1. `lib/landrec.js`: This contains the chaincode logic which reads and writes values to key value-based data store (state db). This defines valid transactions on the ledger.

2. `index.js`: This is used to expose the landrec chaincode module.

3. `docker-compose.yml`: This defines the `fabric-tools` container from which we will deploy our chaincode. This container will access the chaincode as mounted volume and will connect to same docker network as our fabric network components.

Environment for Step 2

```
'use strict';

const { Contract } = require('fabric-contract-api');

class LandRec extends Contract {

    async initLedger(ctx) {
        console.info('============= START : Initialize Ledger ===========');
        const lands = [
            {
                address: '4545 45th St E, Redmond WA, 98052',
                location: '[{lat: 3534.3, lon: 2423}, {lat: 3534.3, lon: 2341}, {lat: 3534.3,
                owner: 'John',
            },
            {
                address: '356 8th St E, Redmond WA, 98052',
                location: '[{lat: 3534.3, lon: 2423}, {lat: 3534.3, lon: 2341}, {lat: 3534.3,
                owner: 'Jane',
            },
            {
                address: '999 156th Ave NE, Redmond WA, 98052',
                location: '[{lat: 3534.3, lon: 2423}, {lat: 3534.3, lon: 2341}, {lat: 3534.3,
                owner: 'Huma',
            },
        ];

        for (let i = 0; i < lands.length; i++) {
            lands[i].docType = 'land';
            await ctx.stub.putState('LAND' + i, Buffer.from(JSON.stringify(lands[i])));
            console.info('Added <--> ', lands[i]);
        }
        console.info('============= END : Initialize Ledger ===========');
    }

    async queryLand(ctx, landNumber) {
        const landAsBytes = await ctx.stub.getState(landNumber); // get the land from chainco
        if (!landAsBytes || landAsBytes.length === 0) {
            throw new Error(`${landNumber} does not exist`);
        }
        console.log(landAsBytes.toString());
        return landAsBytes.toString();
    }

    async createLand(ctx, landNumber, address, location, owner) {
        console.info('============= START : Create Land ===========');

        const land = {
            docType: 'land',
            address,
            location,
            owner,
        };

        await ctx.stub.putState(landNumber, Buffer.from(JSON.stringify(land)));
        console.info('============= END : Create Land ===========');
    }
```

```
    async queryAllLands(ctx) {
        const startKey = 'LAND0';
        const endKey = 'LAND999';

        const iterator = await ctx.stub.getStateByRange(startKey, endKey);

        const allResults = [];
        while (true) {
            const res = await iterator.next();

            if (res.value && res.value.value.toString()) {
                console.log(res.value.value.toString('utf8'));

                const Key = res.value.key;
                let Record;
                try {
                    Record = JSON.parse(res.value.value.toString('utf8'));
                } catch (err) {
                    console.log(err);
                    Record = res.value.value.toString('utf8');
                }
                allResults.push({ Key, Record });
            }
            if (res.done) {
                console.log('end of data');
                await iterator.close();
                console.info(allResults);
                return JSON.stringify(allResults);
            }
        }
    }

    async changeLandOwner(ctx, landNumber, newOwner) {
        console.info('============= START : changeLandOwner ===========');

        const landAsBytes = await ctx.stub.getState(landNumber); // get the land from chainco
        if (!landAsBytes || landAsBytes.length === 0) {
            throw new Error(`${landNumber} does not exist`);
        }
        const land = JSON.parse(landAsBytes.toString());
        land.owner = newOwner;

        await ctx.stub.putState(landNumber, Buffer.from(JSON.stringify(land)));
        console.info('============= END : changeLandOwner ===========');
    }

}

module.exports = LandRec;
```

Running the Environment

1. Run Fabric Network

- *As done in step 1*

  When you run the above environment, you might see your previous environment running with all docker containers still up - if that is the

environment running with all docker containers still up - if that is the case you can skip this step. If its a new environment, we need to bring up

the network again on which we will deploy the chaincode. We will bring up the same network as we did in step 1. This time, you can use a script that will execute all the commands we did in step 1. The script is named `exercise-1.sh` and placed inside the `infra-basic-network` directory. Execute the following to run it:

```
cd /usercode/infra-basic-network && ./exercise-1.sh
```

2. Launch Tools Container

Docker compose command to bring up the tools container.

```
cd /usercode/chaincode && docker-compose up -d cli
```

3. Install chaincode

Here we will use `docker exec` command to execute a command in `cli` container. The command is `peer chaincode install` and takes the name, version, path and language of the chaincode it will install on the peer. Note that the path refers to path of chaincode as mounted on the cli/tools container.

If we had multiple peers on our network, we would need to run the same command for all peers.

For full details on peer chaincode tooling visit: https://hyperledger-fabric.readthedocs.io/en/release-1.4/commands/peerchaincode.html

```
docker exec -e "CORE_PEER_LOCALMSPID=Org1MSP" \
            -e "CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/cr
       cli peer chaincode install \
            -n landrec \
            -v 1.0 \
            -p "/opt/gopath/src/github.com" \
            -l "node"
```

4. Instantiate chaincode

Here we will use `docker exec` command to execute a command in `cli` container. The command is `peer chaincode instantiate` and it takes the orderer node address, channel name, chaincode name, chaincode version

orderer node address, channel name, chaincode name, chaincode version,

chaincode language and chaincode initialization args as parameters. These args will be passed to the init method of our chaincode contract.

The last argument here is the `endorsement policy`. Here, the endorsement policy is defined as `OR` between org1 and org2 members. This means, as long as a peer from either org1 or org2 endorses, our transaction proposal will satisfy the endorsement policy.

See the link when you want to understand more about endorsement policies.

Execute the command below to instantiate chaincode (It might take a while).

```
docker exec -e "CORE_PEER_LOCALMSPID=Org1MSP" \
            -e "CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/cr
       cli peer chaincode instantiate \
            -o orderer.example.com:7050 \
            -C mychannel \
            -n landrec \
            -l "node" \
            -v 1.0 \
            -c '{"Args":[]}' \
            -P "OR ('Org1MSP.member','Org2MSP.member')"
```

5. Test chaincode invoke

Here we will use `docker exec` command to execute a command in `cli` container. The command is `peer chaincode invoke` and takes the orderer, channel, chaincode name to invoke and a string with function name and arguments to invoke in the chaincode.

This is the cli command to invoke chaincode. In actual systems we would need to invoke chaincode from other system components. The Hyperledger Fabric provides SDKs in different languages to do this. In next step, we will use node sdk to invoke chaincode deployed on fabric network.

```
docker exec -e "CORE_PEER_LOCALMSPID=Org1MSP" \
            -e "CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/cr
       cli peer chaincode invoke \
            -o orderer.example.com:7050 \
            -C mychannel \
            -n landrec \
            -c '{"function":"initLedger","Args":[]}'
```

In the next chapter, we will create a full-fledged client application.

Application Integration

In this lesson, we will discuss the two important objects for interaction of node SDK.

WE'LL COVER THE FOLLOWING

- The Wallet
- The Gateway

Applications can interact with a chaincode deployed on a fabric network using a fabric SDK. Hyperledger Fabric 1.4 provides official SDK in node and java.

In this course we will see how to interact with the network using the node SDK.

The node SDK has two important objects for interaction with a fabric network:

- Gateway
- Wallet

The Wallet

The wallet provides storage and usage histories of user identities for transactions on the network. Remember, in our basic-network we deployed fabric-ca. The self-signed certificate for fabric-ca is configured on the network as a trusted root CA so only users that hold a valid certificate & corresponding key from our fabric-ca will be able to submit a transaction on the network. In our example code, we will be storing these keys and certificates in a file-based wallet to keep things simple to understand.

The Gateway

The Gateway provides methods to connect to a network channel, using a user's id from their wallet, and executes chaincode methods installed on the

channel.

```
        // Create a new gateway for connecting to our peer node.
        const gateway = new Gateway();
        // Assuming the wallet has an identity saved by name user1
        // The cpp object contains all network's connection urls
        await gateway.connect(ccp, { wallet, identity: 'user1', discovery:
 { enabled: false } });

        // Get the network (channel) our contract is deployed to.
        const network = await gateway.getNetwork('mychannel');

        // Get the contract from the network.
        const contract = network.getContract('fabcar');

        // Evaluate the specified transaction.
        // queryCar transaction - requires 1 argument, ex: ('queryCar', 'C
AR4')
        // queryAllCars transaction - requires no arguments, ex: ('queryAl
lCars')
        const result = await contract.evaluateTransaction('queryAllCars');
```

In the next lesson, we will discuss about Wallet and Identities.

Wallets and Identities

In this lesson, we will see how wallets and identities are used in client applications through a sequence diagram.

WE'LL COVER THE FOLLOWING

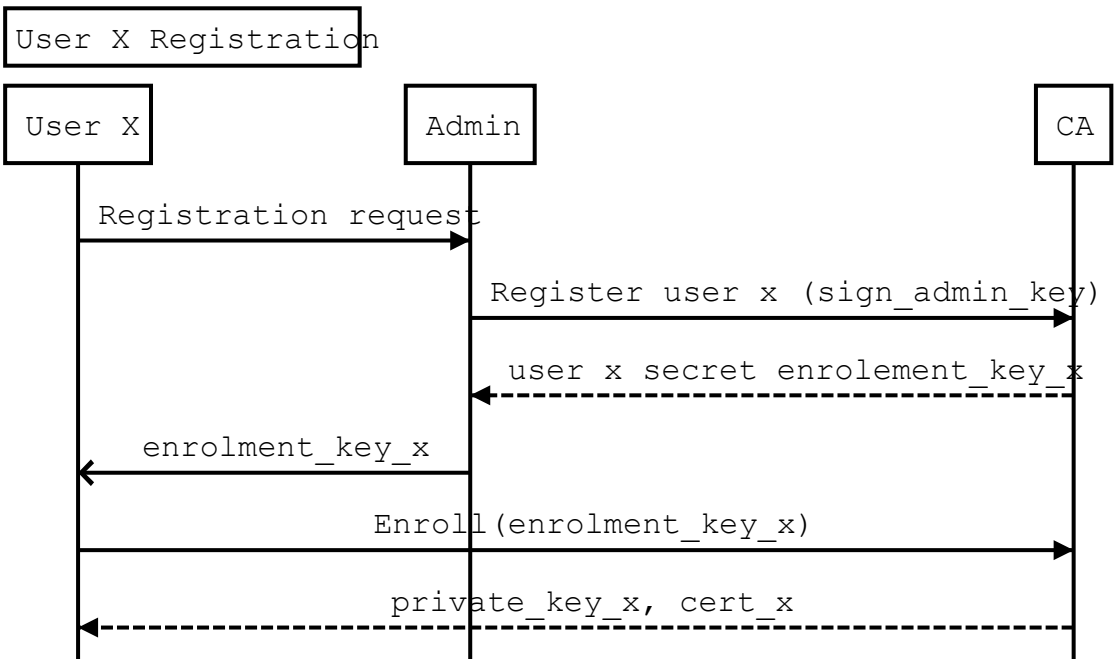
- Wallets and Identities in Client Application
  - First User (Admin)

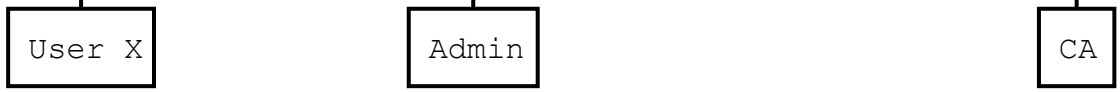
Wallets and Identities in Client Application

Now lets see how the wallet and user identities work.

We can issue a new user's key and get CA signed cert by contacting the CA. Here are the needed steps:

1. A valid existing user A `registers` a new user X to CA. The CA returns the `enrollment key`.
2. The user A shares that `enrollment key` with X
3. User X/A contacts the CA to `enroll` user X and passes in the `enrollment key`. CA returns valid key and signed certificate for user X.





First User (Admin)

When deploying the CA we need to bootstrap it with one initial admin user that can further register new users. The `enrollment key` for this admin user is pre-set in our initial configuration and the admin user is hence already registered when CA starts. Hence, all we need to do is to `enroll` the admin user with right `enrollment key` on application first launch.

In the next lesson, we will deploy our client application.

Deploy Client Application

In this last lesson, we will first see the folder structure of the application and then we will see how to run the environment.

WE'LL COVER THE FOLLOWING

- Folder Structure for API
- Environment for STEP 3
- How to Run the Environment
  - 1. Run Fabric Network and deploy chaincode on it.
  - 2. Start the API server
  - 3. Test your application
- Explore Further
  - Self-practice Exercise

Folder Structure for API

Now lets look at important files inside the `api` folder.

1. `src/index.js`:
   Creates an express.js server that exposes following endpoints.
   - EnrollAdmin
     - This will enroll the admin with CA using SDK and then store Admin's credentials in a file-based wallet.
   - Query
     - This will query the blockchain using the landrec chaincode. It will use fabric node sdk to do so.
   - Invoke
     - This will invoke the `createLand` method in our `landrec` chaincode

2. `src/controllers/*.js`:
   Implementation of all above endpoints in their respective files.

3. `src/controllers/wallet/`:
   Directory in which user identities will be stored.

4. `src/network-connection/connection.json`: Connection configuration file used by fabric node sdk to connect to the blockchain network.

Environment for STEP 3

How to Run the Environment

1. Run Fabric Network and deploy chaincode on it.

When you run the above environment, you might see your previous environment running with all docker containers still up and chaincode deployed. If that is the case you can skip this step. If its a new environment, run the following command.

```
cd /usercode/infra-basic-network && ./exercise-1.sh && cd /usercode/chaincode && ./exercise-2
```

2. Start the API server

```
cd /usercode/api && npm install && node src/index.js
```

3. Test your application

Click your application url now! Click the url shown above the terminal tab - below the code.

- Visit the `/enrollAdmin` endpoint first so admin user's credentials are generated
- Visit the `/query` endpoint to view all data that was put in ledger by our chaincode init method

chaincode init method

- Visit the `/invoke` endpoint to enter new data into ledger
- Visit the `/query` endpoint to verify that new data has been added to ledger

---

Explore Further

Now stop the server and view the files generated in `controllers/src/wallet` folder. These files were generated when you executed the `/enrollAdmin` endpoint.

Self-practice Exercise

Use the environment above to expose API that allows you to transfer ownership of one land to another. Some code is already part of environment that you can re-use.

Bonus: Only the current owner or admin should be allowed to transfer the ownership of a land.

> **HINT:**

- You can add extra attributes to the client certificate when registering a user with CA.
- In chaincode you can fetch the attributes using ClientIdentity class
- Here is how to get a reference to client identity object in chaincode:

```
 * let cid = new ClientIdentity(stub); // "stub" is the ChaincodeStub object passed to Init() and Invoke() methods
 * if (cid.assertAttributeValue('hf.role', 'auditor')) {
 *    // proceed to carry out auditing
 * }
```

This hands-on activity hopefully gives you a good start-to-finish understanding of a typical blockchain based solution.

Summary

Overall course summary

WE'LL COVER THE FOLLOWING

- Next Steps

I hope this course has helped you thoroughly understand how Blockchain based systems work and why are they needed.

Next Steps

To explore Hyperledger Fabric further, I suggest you to clone the fabric-samples repository and try out all the samples they have provided.

Thank you for your time. I hope you have enjoyed this course as much as I have enjoyed making this course.